



中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

学 校	浙江财经大学
--------	--------

参赛队号	20260045613
------	-------------

队员姓名	1.	陈俊杰
	2.	陈振宇
	3.	林晓宇

中国研究生创新实践系列大赛

摘要

大脑活动记录技术中，非侵入式脑电图（EEG）在脑机接口与精神疾病临床诊断中展现出重大应用前景。针对前额叶电极（Fz、F3、F4）极易受高幅眼动（EOG）等生理杂音干扰，且常规滤波极易破坏反映视物形状微弱神经特征的核心难题，本文围绕视觉认知两阶段加工机制，构建了集特征保护型残差自适应去噪、微观-介观-宏观跨尺度脑电计算模型以及多脑区协同认知网络于一体的完整理论与算法体系。

针对问题一，针对前额通道信噪比极低且设备自带滤波严重破坏视物特征的痛点，设计了**特征保护型残差双分量自适应校正与三次 B 样条拟合模型**。首先，通过 60 Hz 陷波与 0.1–30 Hz 零相位四阶双向带通实现保相滤波，结合硬件截幅、极差与物理跃变等多维工程准则进行坏试次客观筛查；其次，采用五折分层交叉验证框架，从同向低伪影试次中构建中位数基底模板并分离单试次残差，解耦提取垂直瞬目与水平扫视正交分量；引入基于稳健中位数绝对偏差（MAD）的软门控激活机制与带边界约束的岭回归模型估计耦合系数，并创新性提出“**训练均值保护（Training Mean Protection）**”机制，在滤除试次间剧烈波动的同时严格约束平均诱发波形的系统性偏离；最后，采用内部结点间隔 50 ms 的三次 B 样条平滑拟合 250–500 ms 认知窗内的正向诱发电位（P300）。实测数据表明：相对预处理基线，本模型使代理 MAE 点估计下降 5.4%–10.0%（受试者 A 项目一降幅达 10.0% [4.9%, 13.8%]）；信噪比代理量平均提升 0.03–0.48 dB；视物形状左右差分幅度保留比达到 0.813–0.952，差分波形相关性均值高于 0.99；在半合成人工伪影注入闭环验证中，本模型在强污染下将归一化 RMSE 显著降低 11.5%，兼具优异的去噪能力与高度的特征保真性。

针对问题二，针对视觉刺激如何跨尺度传导至头皮电极的机理阐释难题，构建了**图像驱动的三级兴奋-抑制（E/I）神经元群体计算模型**。首先，基于 COSFIRE 空间边缘组合思想提取左右三角形的几何轮廓特征，建立对偶镜像偏好输入；其次，采用 Wilson–Cowan 型非线性动力学方程刻画“LGN 中继 → 早期视皮层 → 形状整合 → 前额响应”的级联演化，并以突触等效电流偶极子结合岭正则化优化求解前额头皮（Fz/F3/F4）的有效观测矩阵 $\hat{L}$ ；进而通过固定参数消融与重拟合检验，定量揭示了左右三角产生的脑电波形差异本质上源于几何轮廓非对称性在皮层网络中的非线性传导以及跨半球投射偏侧化。最后，构建融合时域统计、峰潜伏期、偏侧化指数（LI）、机制模板投影系数及残差 RMS 的 **23 维机制增强特征向量**。在严格的五时间块独立留出测试中，方向差分指标 S_{Δ} 在两组数据中显著优于零差基线（受试者 B2 达 0.2562），跨受试者迁移判别平衡准确率最高达 62.31%（AUC 为 0.5807），为空间视觉线索的神经机制解释与脑电特征表征提供了科学自洽的计算模型支撑。

目录

一、 问题重述	3
1.1 问题背景	3
1.2 问题重述	3
二、 问题分析	4
2.1 问题一的分析	4
2.2 问题二的分析	4
2.3 问题三的分析	5
三、 模型假设与符号说明	6
3.1 模型假设	6
3.2 符号说明	6
四、 模型建立与求解	7
4.1 问题一模型的建立与求解	7
4.1.1 数据预处理	7
4.1.2 评价指标	8
4.1.3 问题求解与模型建立	10
4.2 问题二模型的建立与求解	14
4.2.1 建模思路	14
4.2.2 分层兴奋-抑制神经动力学模型的建立	16
4.2.3 脑电信号差异的模型解释	19
4.2.4 视觉刺激响应的特征表示	22
4.3 问题三模型的建立与求解	24
4.3.1 建模思路	25
4.3.2 视觉认知相关信号的截取	26
4.3.3 视觉-记忆-认知宏观模型与脑电差异解释	27
4.3.4 认知状态驱动三类应答模型	30
4.3.5 模型求解结果与验证	32
五、 模型优缺点评价	34
5.1 问题一模型评价	34
5.2 问题二模型评价	34
5.3 问题三模型评价	34
参考文献	35
附录	36

一、 问题重述

1.1 问题背景

大脑活动的记录方式主要分为侵入性和非侵入性两类。皮层脑电图虽能获得较高质量的神经电活动信息，但需要植入电极，存在手术风险和长期稳定性不足等问题。脑电图（EEG）直接在头皮表面采集信号，具有安全、便捷和时间分辨率高等特点，因而在脑机接口、认知研究及神经精神疾病辅助诊断中具有较高的应用价值。

脑电图记录的是大量神经元群体活动经脑组织、颅骨和头皮传导后的叠加结果。它一方面保留了视觉、运动和记忆等神经活动的信息，另一方面也混有眼动、眨眼及设备噪声等干扰。视觉刺激引起的有效响应通常较弱，若降噪过强，刺激形状所对应的波形差异也可能被削弱。因此，如何兼顾伪影抑制与有效特征保留，是脑电数据处理中的首要问题。

在视觉认知过程中，外界刺激经视网膜进入外侧膝状体核（LGN），再投射到大脑皮层，并通过神经群体的协同活动形成头皮电位。不同形状和方向的刺激会激活不同的空间特征组合，但头皮脑电只是多个内部源混合后的宏观观测，难以直接反推出各脑区的活动状态。通道数量有限时，既要解释刺激差异的形成过程，又要避免把数学拟合简单等同于真实脑源定位。

视觉认知还包含从看见刺激、调用记忆到形成判断的连续过程。前额叶与认知加工有关，海马结构承担记忆功能，LGN 负责视觉信息中继。实验中的应答只反映这一过程的外部结果，不保证正确，也可能出现未及时应答的情况。因此，认知模型还应对不同应答状态给出统一表示。

在此背景下，根据视觉任务的多通道脑电记录，建立刺激输入、神经群体活动与头皮观测之间的计算联系，并描述视觉信息的保持、辨认与应答过程，有助于理解脑电波形的产生机理，也可为非侵入式脑机接口及神经精神疾病的量化研究提供参考。

1.2 问题重述

基于上述背景，题目给出两名受试者在两类视觉任务下的四组脑电记录。实验在 Fz、F3 和 F4 三个位置采集原始信号，同时记录视觉提示、目标应答和时间戳。围绕视觉响应的提取、形成机理的解释以及认知过程的刻画，需要解决以下三个问题：

问题 1：脑电信号预处理与有效视觉响应提取

针对目标位置已知的实验项目 1 和目标位置未知、仅给定目标形状的实验项目 2，使用 Fz、F3 和 F4 三个原始通道设计预处理方法。在减弱运动、眨眼等伪影的同时，尽量保留与左右视觉刺激及其形状有关的特征；进而提取各观测点的有效视觉响应，并拟合相应波形。设备处理通道可能损伤视觉特征，故不能将其直接作为分析输入。

问题 2：脑电信号形成机理与视觉刺激区分

根据视觉信息由 LGN 传入皮层并最终形成头皮电位的过程，建立脑电信号的计算模型，说明微观刺激特征、皮层神经群体响应与宏观观测之间的联系。结合物体识别依赖空间特征组合的神经生理学事实，解释左、右三角形刺激产生不同脑电响应的原因，并给出能够表征和区分两类刺激的特征。

问题 3：视觉认知宏观模型的建立与验证

从实验记录中截取与视觉认知有关的脑电片段，其终点可设在应答前约 100 ms。以问题 2 中的信号形成机制为基础，考虑海马体的记忆作用和 LGN 的视觉输入，建立由视觉提示、信息保持、目标辨认到应答形成的宏观认知模型，并利用四组实验数据检验。对于错误应答、应答缺失或未及时应答的试次，应在模型和统计处理中分别作出说明。

二、 问题分析

2.1 问题一的分析

本问题的核心任务是从两类视觉认知实验的原始脑电中提取有效响应，并分别拟合 F_z 、 F_3 、 F_4 三个观测点的响应曲线。本质上，这是一个强干扰背景下的弱信号提取问题：既要减弱眼动、眨眼和运动等伪影，又要保留左右三角形刺激所对应的波形差异。

难点一：视觉响应与生理伪影相互混合，且两个实验项目的信号形态并不相同。

三个电极均位于前额区，眼动和眨眼会在同一位置产生较强电位变化，运动伪影与缓慢漂移也可能落在视觉响应所在的时段内。仅依据幅值大小或某一频段进行滤波，难以将这些成分与有效响应完全分开。项目 1 中受试者已知目标位置，项目 2 中受试者只知道目标形状，两种任务下的准备状态和应答过程存在差异，若统一平均处理，容易掩盖项目间的真实变化。

针对难点一：建立以视觉提示为基准的分项目预处理与伪影校正流程。

首先由视觉提示通道确定事件起点，对 F_z 、 F_3 、 F_4 原始记录完成工频抑制、带通滤波和刺激前基线校正，并用固定规则剔除饱和、突跳等严重异常试次。随后分别在两个实验项目内估计响应参考和伪影校正参数，按提示方向形成事件相关平均波形。校正参数只由训练试次确定，再用于对应的留出试次，以减少同一试次同时参与参数估计和效果评价所造成的偏差。

难点二：去噪效果与视觉特征保留之间存在矛盾，且缺少独立的无噪声真值。

加强平滑或扩大校正幅度可以降低试次间波动，却也可能改变左右条件差异、正向峰值及其潜伏期。题目没有提供同步眼电或独立的无噪声脑电，设备处理后的通道又可能损伤视觉特征，不能作为绝对真值。因此，波形变得平稳并不等于有效信息得到保留，也不能仅凭三个额区电极认定观察到了中央-顶区的典型 P300。

针对难点二：从噪声抑制、条件差异和曲线拟合三个方面联合评价。

在相同试次上比较处理前后的基线波动、峰峰值和试次间离散程度，同时考察左右条件差异、特征时窗内的正向活动及 F_3 、 F_4 的空间差异。再通过人工加入眨眼样、扫视样和运动样扰动，检查校正后波形对污染前记录的恢复程度。最后采用固定复杂度的样条曲线拟合各条件平均响应，以拟合残差、峰值位置和正面积共同描述有效视觉响应，避免用单一指标判断去噪优劣。

2.2 问题二的分析

本问题要求说明左右三角形刺激如何经视觉通路形成可观测的脑电差异，并给出区分两类刺激的特征表示。与问题一侧重信号处理不同，本问题需要把图像中的方向信息、皮层神经群体活动和头皮电位联系起来，属于由刺激到观测的跨尺度建模问题。

难点一：刺激形状与头皮脑电处于不同层次，内部神经活动无法直接观测。

三角形的左右朝向属于图像空间结构， F_z 、 F_3 、 F_4 记录的却是多个活动源经过体积传导后的叠加结果。有限通道下的反演具有非唯一性，即使模型能够拟合实测波形，也不能据此唯一确定每个内部状态所对应的真实脑区。因此，模型既要保留视觉通路的基本生理含义，又要能够从刺激出发生成可与实测数据比较的预测波形。

针对难点一：按照“形状编码—LGN 中继—皮层群体—头皮观测”建立模型链。

首先提取左右三角形的边缘方向及空间组合关系，构造具有镜像差异的形状输入；再利用 LGN 环节完成视觉信息中继，将输入传递至具有形状选择性的兴奋-抑制神经群体。由群体活动形成等效突触源，并通过统一的头皮观测矩阵生成三个额区通道的预测脑电。左右条件共享动力学参数和观测参数，使两类波形的差异主要由刺激结构产生；同时设置去除形状编码、递归连接或侧化观测等对照模型，检查各环节的实际作用。

难点二：平均波形能够拟合，不代表未知方向的单试次可以稳定区分。

左右刺激的共同响应通常强于两者之间的差异，模型可能较好地重现平均波形，却无法稳定预测较弱的方向信息。问题一的部分处理利用了已知提示方向，若直接将这类输出用于未知方向判别，会使特征中预先含有待预测标签。此外，相邻试次可能共享缓慢漂移或实验状态，普通随机划分容易高估判别效果。

针对难点二：将信号形成模型的检验与方向判别分开进行。

形成模型主要比较共同响应、左右差异波和留出试次预测误差，用于判断所建模型能否解释信号的产生过程；方向判别则从原始三通道记录重新构造时间块，只在训练块内完成预处理参数估计、特征标准化和判别器训练。特征包括时域幅度、峰值潜伏期、额区空间差异及机制模板投影系数，并通过留出时间块和块内置换检验判别结果，从而避免把波形拟合能力与单试次识别能力混为一谈。

2.3 问题三的分析

本问题需要把视觉提示、目标出现和应答前的脑电活动联系起来，建立视觉认知过程的宏观模型。相较于前两问，分析时间由短时视觉响应扩展到提示保持、目标辨认和应答形成的连续过程，同时还要处理错误应答和未及时应答等不完整行为信息。

难点一：不同实验项目的事件标记含义不完全一致，认知信号的终点难以统一确定。

项目 2 记录了独立点击，而项目 1 中的部分应答时刻只能由目标标记的变化推定，两类标记的证据强度不同。若分析窗口延伸到应答之后，容易混入运动相关活动；若把没有明确点击的试次一律视为超时，又会人为补充题目未给出的行为标签。除此之外，记录边界、下一次提示和异常波形也会限制单个试次可使用的时间范围。

针对难点一：先核对事件证据，再按应答状态制定统一的截窗规则。

首先核对四组记录中的视觉提示、候选目标和点击变化，将独立点击、应答代理和应答不可观测的试次分别标识。存在可用应答时，取应答前约 100 ms 作为分析终点；没有明确应答时，采用预先规定的有限观察窗，并同时检查记录边界、下一提示和试次质量。错误应答不预先剔除，缺失点击也不直接记为超时，使截窗规则与行为结论彼此分离。

难点二：记忆与认知活动难以直接定位，且数据缺少完整的正确、错误和超时真值。

题目要求考虑海马体和 LGN 等潜在来源，但三个额区电极不能直接定位深部结构。提示后的缓慢变化既可能与记忆保持有关，也可能来自背景漂移或任务准备。与此同时，现有记录没有给出完整的目标布局、正确目标映射和实验超时阈值，点击方向不能直接换算成答题正误，缺失点击也不能直接换算成真实超时。

针对难点二：建立功能性认知状态，并将脑电验证与行为仿真分开处理。

继承问题二的形状编码、LGN 中继和头皮观测结构，增加提示保持、目标触发提取与认知整合状态，构成视觉-记忆-认知的宏观模型。模型先在留出时间块上比较预测波形，并与不含记忆状态的普通慢趋势模型进行对照，判断新增环节是否带来额外解释。随后将认知状态输入证据累积过程，在给定的模拟目标和观察期限下产生正确、错误及未决三类结局，用于研究参数变化对模型行为的影响；实测脑电与模拟行为分别报告，不把仿真结果写成受试者的真实正确率或超时率。

三、 模型假设与符号说明

3.1 模型假设

为突出问题本质并保证模型可求解，在不改变实验含义的前提下作如下基本假设：

假设一：同一受试者、同一实验项目和同一刺激方向下，重复试次具有稳定的条件诱发响应，其余与刺激无关的随机扰动经多次平均后趋于零。

假设二：原始脑电可视为刺激相关活动、背景脑电与生理伪迹的叠加；有限幅度的伪迹校正能够减弱干扰，且不系统改变条件平均响应的主要形态。

假设三：左、右三角形除朝向外具有相同的视觉属性；同一记录内两类刺激共享 LGN 中继、皮层群体动力学及头皮观测结构，其差异主要由形状输入与偏侧化投射产生。

假设四：视觉提示、目标出现与应答过程在时间上可分段描述；模型状态表示相应脑区的功能性群体活动，事件代理仅用于确定分析时窗，不直接等同于真实正误或超时标签。

3.2 符号说明

表 1 本文的符号说明

符号	说明	单位
r	数据记录编号，取值为 A1、A2、B1 或 B2	无
i	单组脑电记录中的试次编号	无
c	脑电观测通道编号，取值为 Fz、F3 或 F4	无
f_s	脑电采样频率， $f_s = 256$	Hz
t	相对于视觉提示起点的时间	s
d_i	第 i 次提示的方向，左、右分别记为 $-1, +1$	无
$x_{r,i,c}(t)$	第 r 组记录中第 i 个试次在通道 c 上的原始脑电	原始单位
$\tilde{x}_{r,i,c}(t)$	特征保护型方法校正后的单试次脑电波形	原始单位
$\bar{x}_{r,d,c}(t)$	记录 r 、方向 d 、通道 c 的条件平均响应	原始单位
I_d	朝向为 d 的标准化三角形图像输入	无
$g(t)$	LGN 的中继状态	模型单位
$E_s(t)$	第 s 级皮层兴奋性群体状态	模型单位
$I_s(t)$	第 s 级皮层抑制性群体状态	模型单位
ϕ_i	用于提示方向判别的单试次机制增强特征向量	混合
$z(t)$	行为仿真中的累积决策变量	无

四、 模型建立与求解

4.1 问题一模型的建立与求解

为解决原始额区脑电中视觉响应微弱、瞬目与扫视伪影幅值较大且常规强滤波容易改变条件波形的问题，本文建立特征保护型残差双分量自适应校正模型。模型以原始 F_z 、 F_3 、 F_4 三通道及视觉提示事件为输入，依次完成连续信号保相滤波、坏试次筛查、折外代理模板构建、双分量伪影估计、保守校正、条件事件相关电位提取与三次 B 样条拟合，最终输出两个实验项目在三个观测点的左右刺激响应曲线。设备自带处理通道仅用于可视化对照，不参与建模与评价。

4.1.1 数据预处理

(1) 数据通道与事件审计

题目给出受试者 A、B 在项目一和项目二下的四组数据，采样率均为 $f_s = 256$ Hz。本文只读取原始通道 F_z 、 F_3 、 F_4 和视觉提示通道 VisCue；其中 VisCue 从 0 跳变为 -1 或 $+1$ 的时刻分别记为左、右三角形提示起点。以提示起点 t_i 为零时刻，截取区间 $[-250, 796.875]$ ms，并将 250–500 ms 作为题目给定的主要视觉响应分析窗。由于数据未提供可信的物理标定信息，全文幅值均记为“原始电位单位”，不擅自换算为 μV 。

设备自带的 FzDecon、F3Decon、F4Decon 已改变原始波形的低频趋势，不能作为无噪声真值。图 4.1 给出同一试次三通道原始记录、设备处理结果与本文预处理及校正结果的对照。可以看出，设备处理曲线与原始曲线存在明显形态差异，因此后续方法完全从原始三通道重新计算。

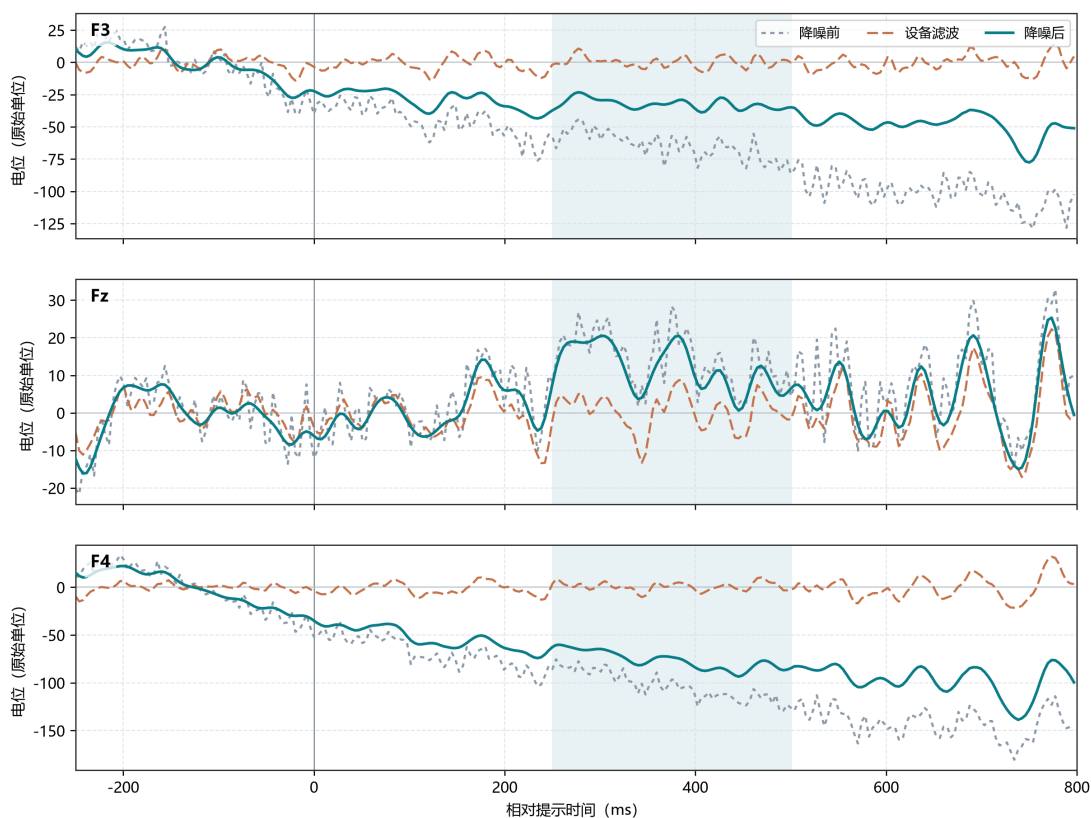


图 4.1: 同一试次三通道原始记录、设备处理与本文处理结果对比

(2) 连续信号保相滤波与基线校正

首先对连续信号施加中心频率为 60 Hz、品质因数 $Q = 30$ 的二阶 IIR 陷波器，以削弱工频干扰；随后采用 0.1–30 Hz 四阶 Butterworth 带通滤波器。两类滤波均以前后双向方式执行，使总体相位响应为零，从而避免因因果滤波时延改变峰潜伏期。记陷波与带通算子分别为 $\mathcal{F}_{\text{notch}}$ 和 $\mathcal{F}_{\text{bp}}$ ，则滤波信号为

$$x_c^{(f)}(t) = \mathcal{F}_{\text{bp}}(\mathcal{F}_{\text{notch}}(x_c^{(0)}(t))), \quad c \in \{F_z, F_3, F_4\}. \quad (1-1)$$

完成事件分段后，以刺激前 $[-250, 0)$ ms 的中位数进行稳健基线校正：

$$x_{i,c}(t) = x_{i,c}^{(f)}(t) - \text{median}_{u \in [-250, 0)} x_{i,c}^{(f)}(u), \quad (1-2)$$

其中， i 为试次编号， c 为通道， t 为相对提示时间。选择 0.1 Hz 低截止频率的目的是保留项目中较慢的条件平均变化；若直接提高高通截止频率，虽然可减小漂移，但也可能人为改变宽缓响应的幅值与潜伏期。

(3) 严重坏试次筛查

为避免硬件截幅、极端振幅和非生理跳变支配均值，本文在任何模板估计前固定三项工程阈值。设试次 i 的跨通道饱和点数、最大峰峰值和最大相邻点跳变分别为 S_i 、 P_i 和 J_i ，则坏试次指示量定义为

$$b_i = 1(S_i \geq 15 \vee P_i > 1800 \vee J_i > 600), \quad (1-3)$$

其中， $S_i = \sum_{c,t} 1(|x_{i,c}^{(0)}(t)| \geq 999)$ ， $P_i = \max_c \{\max_t x_{i,c}(t) - \min_t x_{i,c}(t)\}$ ， $J_i = \max_{c,t} |x_{i,c}(t) - x_{i,c}(t-1)|$ 。这些阈值由当前数据量纲下的异常分布与硬件截幅特征确定，只用于剔除不可恢复的极端记录，不把一般的高幅试次预先删除。筛查结果如表 2 所示，四组数据共保留 371 个试次。

表 2: 四组实验数据的事件与有效试次对账

数据组	提示事件数	边界剔除	质量剔除	有效试次	左/右试次
受试者 A 项目一	100	0	10	90	47/43
受试者 A 项目二	100	0	5	95	53/42
受试者 B 项目一	100	0	6	94	46/48
受试者 B 项目二	100	0	8	92	45/47

4.1.2 评价指标

问题一没有同步眼电或独立无噪声脑电，故设备处理波形与训练模板均不能视为真实标签。为同时评价去噪、条件差异保留与曲线拟合，本文采用代理参考误差、信噪比代理量、左右差异保留率、半合成恢复误差和样条拟合误差五类互补指标，而不将其合成为单一分数。

(1) 代理参考误差与信噪比代理量

在第 k 个外折中，仅用训练试次构造方向 $d \in \{-1, +1\}$ 的代理模板 $T_{d,c}^{(k)}(t)$ 。测试集方向平均响应记为 $\bar{x}_{d,c}^{(k)}(t)$ ，则 250–500 ms 窗内的代理平均绝对误差为

$$\text{MAE}_{\text{proxy}} = \frac{1}{|\mathcal{W}|} \sum_{t \in \mathcal{W}} \left| \bar{x}_{d,c}^{(k)}(t) - T_{d,c}^{(k)}(t) \right|, \quad \mathcal{W} = [250, 500] \text{ ms}. \quad (1-4)$$

该指标越小，说明测试条件平均曲线越接近训练折内的低伪影参考，但不能据此断言更接近真实神经信号。进一步以 0–500 ms 内 ERP 功率与单试次相对 ERP 的残差功率之比定义信噪比代理量：

$$\text{SNR}_{\text{proxy}} = 10 \log_{10} \frac{\frac{1}{|\mathcal{W}_0|} \sum_{t \in \mathcal{W}_0} \bar{x}_{d,c}^2(t)}{\frac{1}{N_d |\mathcal{W}_0|} \sum_{i: y_i = d} \sum_{t \in \mathcal{W}_0} [x_{i,c}(t) - \bar{x}_{d,c}(t)]^2}, \quad \mathcal{W}_0 = [0, 500] \text{ ms}. \quad (1-5)$$

残差中仍含真实的试次间变异，因此该指标只反映条件平均响应相对试次波动的突出程度。

(2) 视觉条件差异保留指标

令 $\Delta_c^{\text{pre}}(t)$ 与 $\Delta_c^{\text{V7}}(t)$ 分别表示预处理后和校正后左减右 ERP 差分，则定义差分幅度保留率与形状相关系数为

$$R_{\Delta,c} = \frac{\|\Delta_c^{\text{V7}}\|_2}{\|\Delta_c^{\text{pre}}\|_2}, \quad \rho_{\Delta,c} = \text{corr}(\Delta_c^{\text{V7}}, \Delta_c^{\text{pre}}), \quad (1-6)$$

其中范数和相关均在 250–500 ms 计算。 $R_{\Delta,c}$ 接近 1 且 $\rho_{\Delta,c}$ 较高，仅表示处理前后的条件差异较接近；若原始条件差异含方向相关眼动，该指标同样会保留该成分，因而不是视觉神经特异性的直接证据。

(3) 已知目标恢复误差与曲线拟合误差

在半合成检验中，将人工伪影加入预处理实测背景，以污染前波形 x^* 为可计算目标。归一化恢复误差定义为

$$\text{NRMSE} = \frac{\sqrt{\frac{1}{NCT} \sum_{i,c,t} (\hat{x}_{i,c}(t) - x_{i,c}^*(t))^2}}{\sqrt{\frac{1}{NCT} \sum_{i,c,t} (x_{i,c}^*(t))^2}}, \quad (1-7)$$

该指标可以检验对“额外注入污染”的恢复能力，但污染前实测背景仍可能含原有伪影。对于最终 ERP 的三次 B 样条拟合，另以同一样本上的残差均方根误差 RMSE_{fit} 描述平滑拟合程度，它不是独立预测误差。

(4) 不确定性估计与指标解释

对左右方向分别进行配对试次 Bootstrap 重采样，预处理与 V7 共用同一批索引；组均值指标使用 800 次重采样并报告 95% 区间。图 4.2 汇总了代理 MAE 降幅、SNR 代理增量和左右差异保留率。可见四组代理 MAE 降幅点估计均为正，区间为 5.4%–10.0%；SNR 代理增量仅为 0.027–0.481 dB，且四组区间均跨过零点；左右差异保留率点估计为 0.813–0.952。因此，评价证据支持“校正后更接近训练代理参考且未大幅压缩处理前条件差异”，但不支持“真实神经信噪比已稳定提高”这一更强结论。

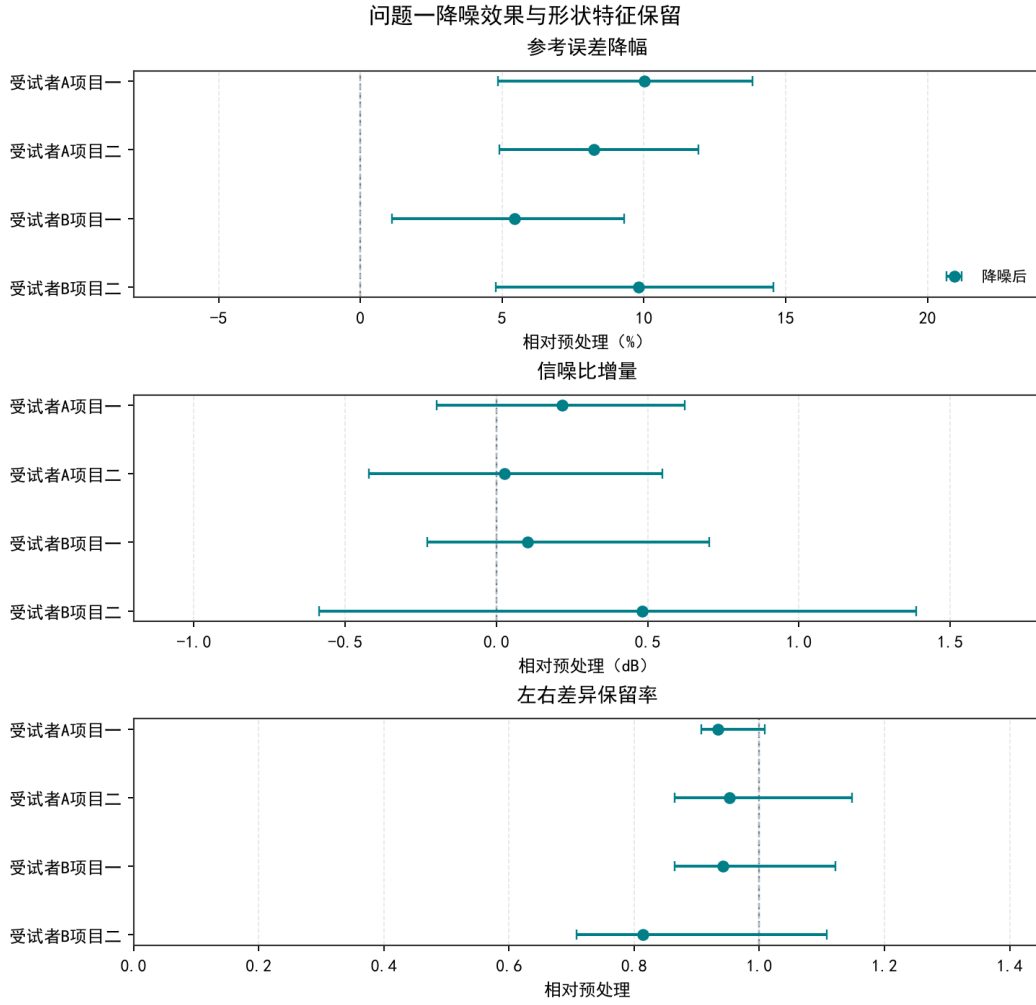


图 4.2: 四组数据的去噪代理指标与左右条件差异保留结果

表 3: 问题一核心评价结果及 95% Bootstrap 区间

数据组	有效试次	代理 MAE 降幅/%	SNR 代理增量/dB	左右差异保留率
受试者 A 项目一	90	10.0 [4.9, 13.8]	0.217 [-0.197, 0.623]	0.934 [0.908, 1.009]
受试者 A 项目二	95	8.2 [4.9, 11.9]	0.027 [-0.422, 0.549]	0.952 [0.865, 1.148]
受试者 B 项目一	94	5.4 [1.1, 9.3]	0.103 [-0.228, 0.702]	0.942 [0.865, 1.121]
受试者 B 项目二	92	9.8 [4.8, 14.6]	0.481 [-0.585, 1.388]	0.813 [0.708, 1.107]

4.1.3 问题求解与模型建立

(1) 五折外层划分与训练代理模板

将每组有效试次按左右方向分层划分为五折, 随机种子固定为 42。对第 k 折, 测试试次不参与模板、残差尺度或候选策略的估计。首先在训练集内按稳健伪影得分从低到高排序, 分别选取左右方向中较低的 50% 试次, 并计算逐时间点中位数:

$$T_{d,c}^{(k)}(t) = \text{median}_{i \in \Omega_{d,\text{low}}^{(k)}} x_{i,c}(t), \quad d \in \{-1, +1\}. \quad (1-8)$$

该模板的作用是从单试次中分离相对于同方向条件平均的异常波动，而不是充当无噪声真值。单试次残差写为 $r_{i,c}(t) = x_{i,c}(t) - T_{d,c}^{(k)}(t)$ 。

(2) 垂直--水平双分量与软门控

前额区瞬目通常在三个导联上表现为较强的同相变化，水平扫视则更容易在 F_3 与 F_4 形成反相差异。基于这一几何结构，从残差构造垂直与水平候选参考：

$$v_i(t) = \underset{c \in \{F_2, F_3, F_4\}}{\text{median}} \ r_{i,c}(t), \quad h_i(t) = \frac{r_{i,F_4}(t) - r_{i,F_3}(t)}{2}. \quad (1-9)$$

分别用约 160 ms 和 120 ms 的 Savitzky-Golay 窗对 v_i 、 h_i 平滑。设训练折估计的逐点 MAD 尺度为 $s_q(t)$ ，门控阈值和过渡宽度为 η_q 、 w ，则软门控分量统一表示为

$$a_q(t) = \tilde{q}(t) \cdot \text{clip}\left(\frac{|\tilde{q}(t)|/(s_q(t) + \varepsilon) - \eta_q}{w}, 0, 1\right), \quad q \in \{v, h\}. \quad (1-10)$$

软门控使低于训练残差尺度的波动基本保持不变，而对超过阈值的缓慢同相或反相成分逐渐增加校正强度，从而减小硬阈值引起的波形突变。

(3) 带边界约束的岭回归校正

将两个门控分量组成设计矩阵 $A_i = [a_v, a_h]$ 。对每个通道分别求解

$$\hat{\beta}_{i,c} = \arg \min_{\beta} \|r_{i,c} - A_i \beta\|_2^2 + \lambda \|\beta\|_2^2, \quad (1-11)$$

其中 $\lambda = 0.05$ 。垂直系数限制在 $[0, 1.8]$ ，水平系数限制在 $[-1.5, 1.5]$ ；由于 F_z 位于中线，其水平系数进一步限制在 $[-0.3, 0.3]$ 。全量候选改动量为

$$\delta_{i,c}(t) = A_i(t) \hat{\beta}_{i,c}, \quad x_{i,c}^{\text{cand}}(t) = x_{i,c}(t) - \delta_{i,c}(t). \quad (1-12)$$

岭正则与系数边界共同限制了共线分量的过度扣除；但由于只有三个额区通道， v_i 、 h_i 仍应理解为伪影候选方向，而不是已经分离出的独立眼电源。

(4) 训练内候选选择与均值保护

每个外折训练集再划分训练子集和验证子集，在验证背景上注入 0、1、2 倍人工污染，对“不校正、中位数基线、保守双分量、仅垂直分量、时域保护”等候选进行比较。候选损失由全波形恢复误差、左右差分恢复误差和正向均值误差组成：

$$\mathcal{L}_\ell = \frac{\text{RMSE}_\ell + 0.5 \text{RMSE}_{\Delta, \ell} + E_{+, \ell}}{\text{RMS}_{\text{background}}}, \quad \mathcal{L} = 0.50 \mathcal{L}_0 + 0.25 \mathcal{L}_1 + 0.25 \mathcal{L}_2. \quad (1-13)$$

若训练内选中“保守双分量”，则对测试试次仅采用 25% 候选改动，并回补同方向训练试次平均改动量的 10%；其余候选仅采用 15% 改动。记 $m_{d,c}^{(k)}(t)$ 为训练集同方向 $\delta_{i,c}(t)$ 的均值，最终输出为

$$\tilde{x}_{i,c}(t) = \begin{cases} x_{i,c}(t) - 0.25 \delta_{i,c}(t) + 0.10 m_{d,c}^{(k)}(t), & \text{保守双分量分支,} \\ x_{i,c}(t) - 0.15 \delta_{i,c}(t), & \text{其他候选分支.} \end{cases} \quad (1-14)$$

该结构把“对单试次异常波动的校正”和“对条件平均形态的保护”分开控制。比例参数来自四组数据的开发过程，外折隔离防止测试试次直接进入自身参数估计，但不能把当前结果称为全新受试者的独立验证。

(5) 条件 ERP 提取与三次 B 样条拟合

汇总五个外折的测试输出后，按刺激方向分别计算条件 ERP：

$$\bar{x}_{d,c}(t) = \frac{1}{N_d} \sum_{i:y_i=d} \tilde{x}_{i,c}(t). \quad (1-15)$$

为获得连续、便于比较的响应曲线，在 50–750 ms 上采用内部结点间隔为 50 ms 的三次 B 样条：

$$f_{d,c}(t) = \sum_{j=1}^M \theta_{j,d,c} B_{j,3}(t), \quad \hat{\theta}_{d,c} = \arg \min_{\theta} \sum_{t=50}^{750} \left[\bar{x}_{d,c}(t) - \sum_{j=1}^M \theta_j B_{j,3}(t) \right]^2. \quad (1-16)$$

固定结点间隔可抑制有限试次平均后的局部毛刺，并避免逐点插值。四组数据、左右方向及三个通道共 24 条曲线在 50–750 ms 上的拟合 RMSE 为 0.497–0.975 原始单位，在 250–500 ms 上为 0.366–1.131 原始单位。图 4.3 展示受试者 A 项目一的代表性拟合结果，灰色曲线为残差；拟合误差较小只说明样条能描述当前平均波形，不等于对新试次的预测性能。

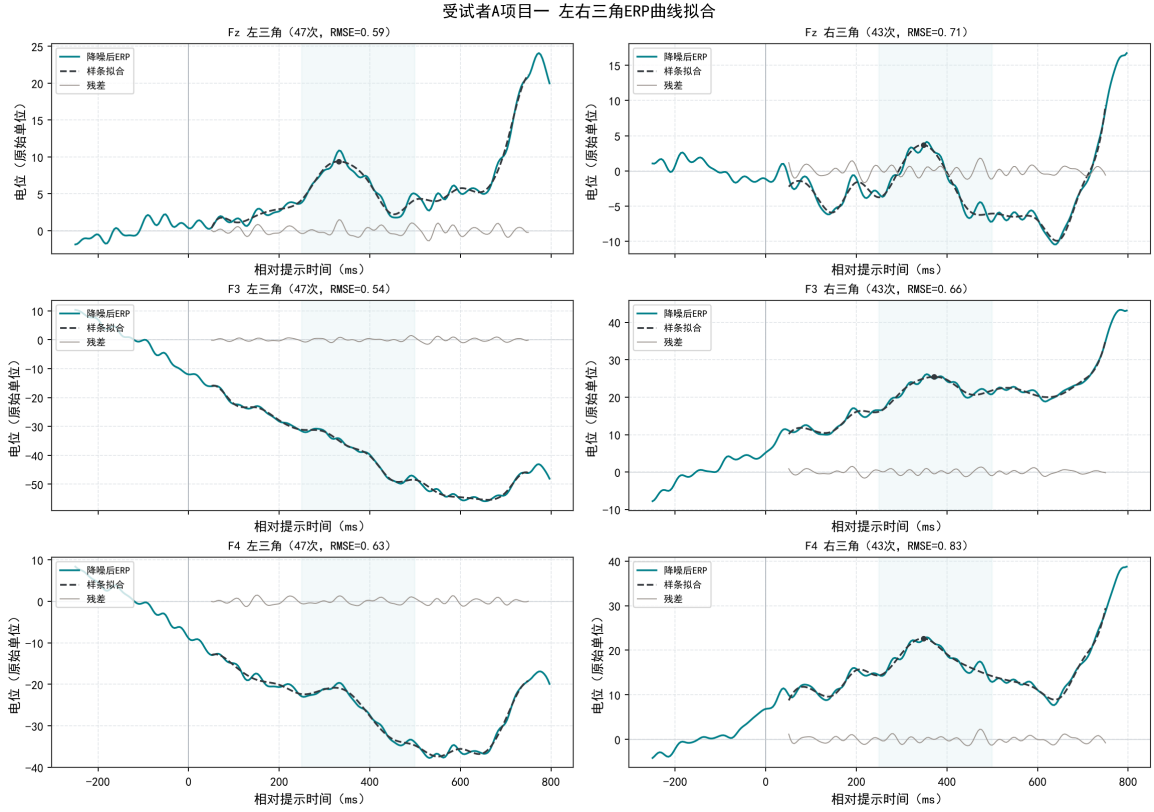


图 4.3: 受试者 A 项目一左右条件 ERP 的三次 B 样条拟合与残差

(6) 求解结果与稳健性检验

四组数据的左右条件平均曲线如图 4.4 所示。实线与虚线分别表示校正后和预处理后 ERP，阴影区为 250–500 ms 分析窗。校正后曲线整体保留了处理前的主要低频形态，同时对部分试次间波动进行了收缩。四组数据的逐通道左右差分波形相关系数均值为 0.992–0.998，但将试次按时间顺序前后对半后，受试者 A 项目一和受试者 B 项目二的三通道差分相关分别为 -0.322 和 -0.400 ，说明条件差异的跨时段稳定性尚未建立。

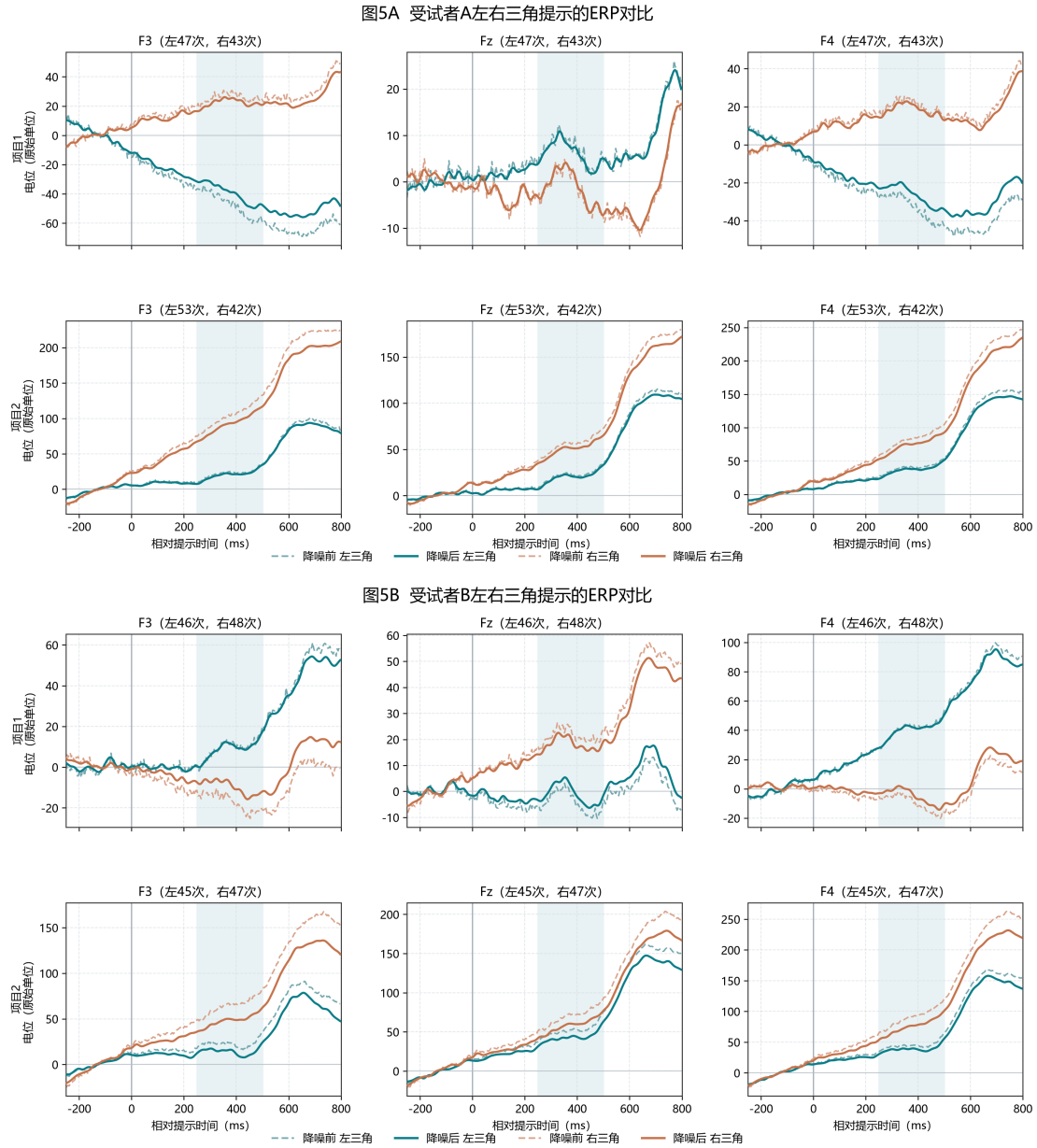


图 4.4: 受试者 A、B 在两个项目中的左右提示三通道 ERP 对比

进一步在真实测试背景上注入不同强度的瞬目样、扫视样和短暂运动样污染，结果如图 4.5 所示。强度为 2 倍时，四组 V7 归一化 RMSE 相对未校正结果分别下降约 9.9%、11.6%、10.8% 和 11.7%；但受试者 A 项目一在 0.5 倍轻扰动下由 0.174 上升至 0.206，且零注入时 V7 仍产生 0.090–0.124 的背景改动。这表明模型更适合抑制较强的附加污染，不能保证每一种低强度场景均获得收益。

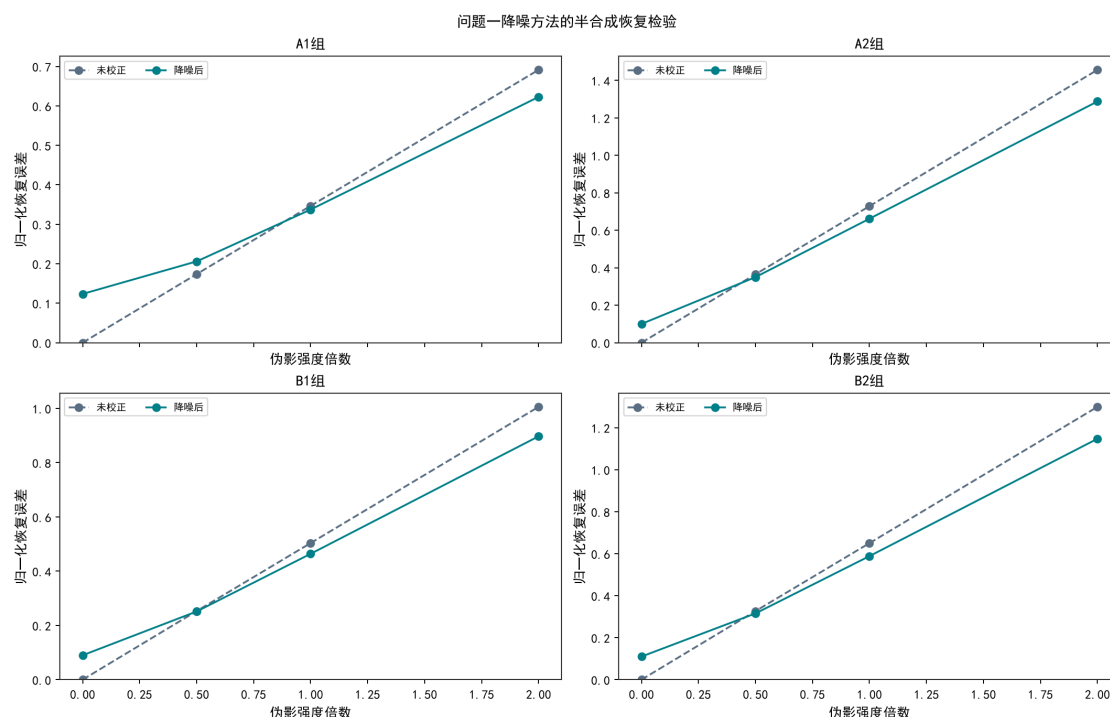


图 4.5: 四组数据在不同伪影强度下的半合成恢复检验

综上，问题一得到了一套可复现的“原始通道预处理—折外自适应校正—条件 ERP 提取—样条拟合”求解方案。现有证据能够说明：在当前四组开发数据上，模型降低了相对训练代理模板的误差，并在较强人工污染下改善了恢复误差；现有证据不能单独证明：被保留的左右差异全部来自视觉神经活动、前额三通道已观测到典型中央—顶区 P300，或该方法能够直接完成未知方向识别。上述校正结果作为已知条件下的有效视觉响应，将为问题二的机理建模提供观测输入。

4.2 问题二模型的建立与求解

问题二要求由视觉刺激出发，建立贯通外侧膝状体核（LGN）、视皮层神经群与头皮脑电的正向计算模型，并据此解释左、右三角刺激所对应的脑电差异，进一步给出可供未知试次使用的特征表示。考虑到题给数据仅包含 Fz、F3、F4 三个前额通道，且未提供个体头模型、独立视觉源定位结果及同步眼动信息，本文不直接把头皮电位差异等同于某一脑区的唯一活动，而是构建“形状编码—群体动力学—突触等效源—条件共享观测”的可检验模型，将机制假设转化为可在独立时间块上验证的波形预测问题。

4.2.1 建模思路

(1) 数据口径与预处理

本问使用受试者 A、B 在项目一和项目二中的四组原始记录，分别记为 A1、A2、B1、B2。全部分析仅使用原始脑电通道 Fz、F3、F4，采样率为 256 Hz；数据文件中的机器处理通道不作为“干净真值”，第 7 通道为心电通道，亦不将其误作眼电回归量。按照事件通道 VisCue 的符号，将 -1 与 +1 分别编码为左三角和右三角刺激。为兼顾刺激前基线和刺激后反应，截取每次提示前 250 ms 至提示后约 800 ms 的脑电片段。

连续记录先按时间顺序划分为 5 个互不重叠的时间块，再在每一块内独立完成 0.1–30 Hz 带通、60 Hz 陷波及提示前中位数基线校正。块边界两侧设置 24 s 保护区，防止双向滤波把测试块信息卷

入训练块。经事件完整性、边界可用性和数值有效性检查后，四组记录分别保留 67、70、69、69 个试次，共 275 个试次，见表 4。

表 4: 问题二四组记录的数据审计与留出划分

记录	受试者	实验项目	有效试次数	连续时间块数
A1	A	项目一	67	5
A2	A	项目二	70	5
B1	B	项目一	69	5
B2	B	项目二	69	5
合计	2 人	2 个项目	275	20

在验证过程中，第 k 个时间块只作为外层测试集，其余四块用于参数选择和观测矩阵估计。任何依赖类别均值、尺度分位数、缺失值填补或判别器参数的统计量，均只在训练块内计算后作用于测试块。由此，模型接受的是一个未知方向的原始试次，而非已经利用该试次标签构造的方向均值。

(2) 总体技术路线

本文采用“先建立可解释正向模型，再提炼可验证特征”的两阶段思路。第一阶段将左、右三角的镜像轮廓转化为三个形状偏好群的输入，经 LGN 中继和三级兴奋-抑制 (E/I) 神经群传播，得到九维突触等效源；随后用左右条件共享的观测矩阵映射到 Fz、F3、F4。第二阶段从单试次中提取时域幅值、峰值与潜伏期、空间侧化和机制模板投影等特征，并在连续时间块留出框架下检验其区分能力。

整体建模链路如图 4.6 所示。上排给出左右三角及两种反事实扰动，用于检查形状编码是否正确由边缘组合驱动；下排展示从视觉边缘、LGN 中继、三级视觉皮层 E/I 神经群到突触源和头皮观测的计算路径。这里的三角图形依据题图示意进行标准化镜像构造，只承担模型输入编码作用，不将其解释为经过屏幕几何标定的真实刺激图像。

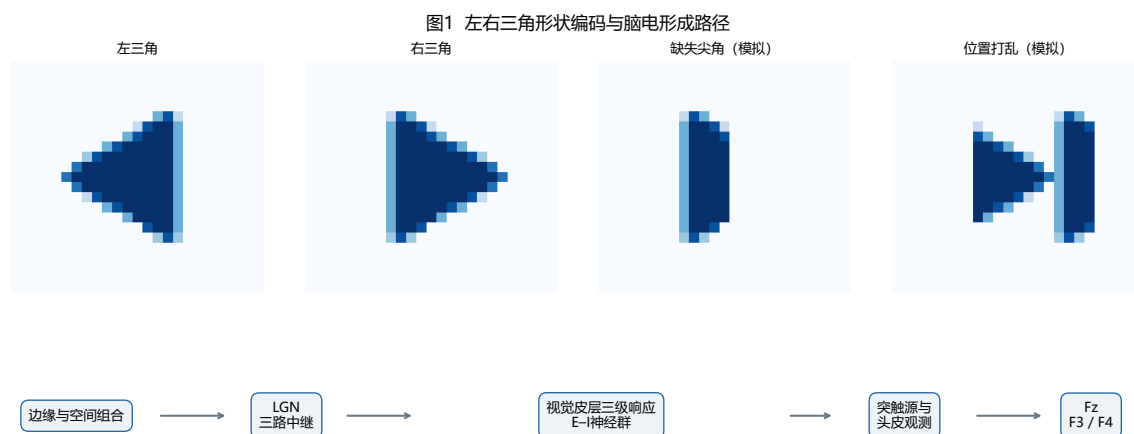


图 4.6: 左右三角形形状编码及脑电正向计算模型的总体路径

4.2.2 分层兴奋--抑制神经动力学模型的建立

(1) 视觉形状的空间边缘组合编码

左、右三角具有镜像对称的轮廓结构。若只使用全图像素差，会使输入对位置和平移过于敏感，因此本文借鉴 COSFIRE 感受野的“定向边缘加空间组合”思想 [6]。设标准化刺激图像为 $X_d(x, y)$, $d \in \{L, R\}$, $r_\theta(p; X_d)$ 表示在位置 p 、方向 θ 上的边缘响应。对形状偏好群 $j \in \{L, R, 0\}$, 在 K_j 个模板点上进行局部容差最大化，再以几何平均聚合：

$$f_j(X_d) = \exp \left[\frac{1}{K_j} \sum_{k=1}^{K_j} \log \left(\max_{\delta \in \mathcal{N}} r_{\theta_{jk}}(p_{jk} + \delta; X_d) + \varepsilon \right) \right], \quad \varepsilon = 10^{-6}. \quad (2-1)$$

其中, L 、 R 分别表示左、右形状偏好群, 0 表示共同边缘群; $\mathcal{N}$ 为局部位移容差邻域。镜像刺激使得 $f_L(X_L) > f_R(X_L)$ 、 $f_R(X_R) > f_L(X_R)$, 但模型后续动力学参数在两种条件间完全共享, 因而左右差异只能来自输入编码, 而不能来自人为设置两套不同模型。

(2) LGN 中继与三级 E/I 神经群

将形状编码向量记为 $\mathbf{f}(X_d) = [f_L, f_R, f_0]^T$ 。LGN 中继状态 $\mathbf{g}(t)$ 接受有限持续时间的视觉驱动：

$$\tau_g \dot{\mathbf{g}}(t) = -\mathbf{g}(t) + 3\mathbf{f}(X_d) \mathbb{I}_{[0, T_s)}(t), \quad T_s = \frac{52}{256} \text{ s}. \quad (2-2)$$

为描述由早期视觉、中阶形状整合至额区响应的时序传递，构建三级 Wilson–Cowan 型神经群模型 [1, 2]。令 $\mathbf{E}_s(t), \mathbf{I}_s(t) \in \mathbb{R}^3$ 分别为第 s 级的兴奋性与抑制性群体状态, $s = 1, 2, 3$, 则

$$\tau_{E,s} \dot{\mathbf{E}}_s = -\mathbf{E}_s + \sigma(\mathbf{E}_s - \mathbf{I}_s + \rho \mathbf{W} \mathbf{E}_s + \mathbf{d}_s - 2\mathbf{1}), \quad (2-3)$$

$$\tau_{I,s} \dot{\mathbf{I}}_s = -\mathbf{I}_s + \sigma(\mathbf{E}_s - \mathbf{I}_s - 2\mathbf{1}), \quad (2-4)$$

$$\mathbf{d}_1 = \mathbf{g}, \quad \mathbf{d}_s = 3(\mathbf{E}_{s-1} - \mathbf{E}_{s-1,0}), \quad s = 2, 3. \quad (2-5)$$

式中, $\sigma(u) = 1/(1 + e^{-u})$ 按元素作用, $\mathbf{E}_{s-1,0}$ 为静息状态; $\mathbf{W}$ 的主对角元为 0、非对角元为 1/2, 用以表示同层不同形状偏好群之间的侧向复发耦合, ρ 为复发强度。三级时间常数逐层增大, 使模型可以表达早期快速感觉响应、中期形状整合及较慢额区响应, 而不把三者压缩为一个无时延的线性映射。

图 4.7 以 A1 为例展示完整模型内部状态。灰色区为视觉驱动时段；从早期视觉层到形状整合层、再到额区响应层，兴奋性状态和突触等效源的峰值逐级后移。蓝、红两线的分离首先出现在与输入方向一致的形状偏好群，并随级联传播至后续层级。这一结果说明模型内部确能把镜像轮廓差异转化为具有时延的动态差异，但它本身仍属于模型结构结果，不能单独证明相应神经群已被实际观测。

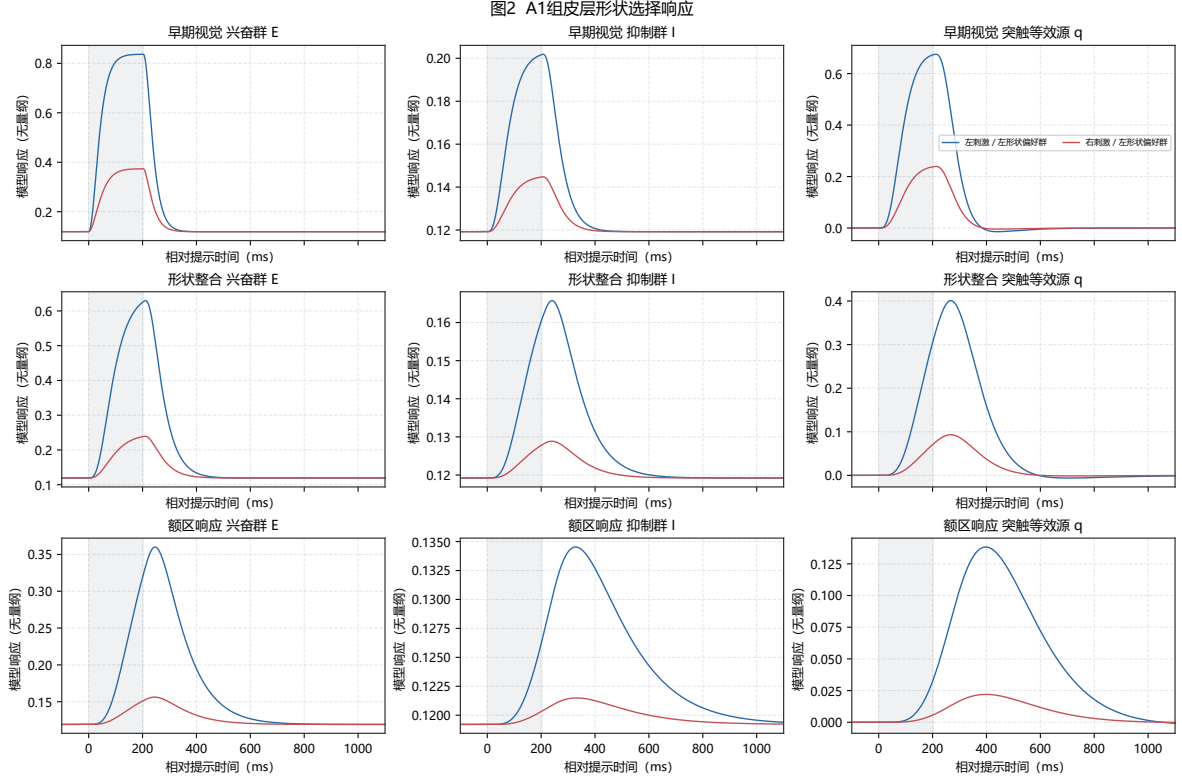


图 4.7: A1 记录对应的三级 E/I 神经群及突触等效源响应

(3) 突触等效源与头皮观测

头皮脑电主要反映大规模同步突触后电流，而非单个神经元动作电位 [3]。因此，对每一级兴奋、抑制状态分别施加二阶低通突触核：

$$\tau_{e,s} \dot{u}_{e,s} = E_s - E_{s,0} - u_{e,s}, \quad \tau_{e,s} \dot{v}_{e,s} = u_{e,s} - v_{e,s}, \quad (2-6)$$

$$\tau_{i,s} \dot{u}_{i,s} = I_s - I_{s,0} - u_{i,s}, \quad \tau_{i,s} \dot{v}_{i,s} = u_{i,s} - v_{i,s}. \quad (2-7)$$

第 s 级的突触等效源定义为

$$q_s(t) = v_{e,s}(t) - 0.7v_{i,s}(t). \quad (2-8)$$

将三级、三群的源活动拼接为九维向量 $q(t) \in \mathbb{R}^9$ ，并施加与实测脑电相同的滤波和基线处理，得到 $\tilde{q}(t)$ 。Fz、F3、F4 的预测向量为

$$\hat{y}_d(t) = L \tilde{q}_d(t), \quad \hat{y}_d(t) \in \mathbb{R}^3, \quad L \in \mathbb{R}^{3 \times 9}. \quad (2-9)$$

由于缺少个体 MRI 和电导率头模型， L 被定义为训练块内估计的“有效观测矩阵”，而非可直接对应解剖位置的物理导联场。为了同时拟合两种条件共有的 ERP 背景与较弱的方向差异，定义

$$C_y = \frac{y_R + y_L}{2}, \quad D_y = y_R - y_L,$$

并对源信号构造相应的 C_q, D_q 。条件共享观测矩阵通过下式求解：

$$\hat{L} = \arg \min_L \left\{ \frac{1}{2} \|C_y - LC_q\|_{W_t}^2 + \frac{1}{2} \|D_y - LD_q\|_{W_t}^2 + \lambda \|L\|_F^2 \right\}. \quad (2-10)$$

时间权重 W_t 对 50–250 ms、250–500 ms 和 500–750 ms 三个分析窗等权赋值，避免样本点数不同使某一时间窗主导目标函数。

(4) 参数选择与数值求解

模型微分方程按 256 Hz 的时间网格数值积分。固定参数用于描述逐层变慢的群体与突触过程；全局时间尺度、复发强度和岭参数则仅在外层训练块中选择，具体见表 5。

表 5: 问题二神经动力学模型的主要参数

参数	含义	候选值或设定	确定方式
τ_g	LGN 中继时间常数	$20 \times \text{scale ms}$	层级时间先验
$\tau_{E,1:3}$	三级兴奋群时间常数	$[20, 40, 80] \times \text{scale ms}$	层级时间先验
$\tau_{I,1:3}$	三级抑制群时间常数	$[40, 80, 160] \times \text{scale ms}$	层级时间先验
$\tau_{e,1:3}$	兴奋性突触时间常数	$[25, 50, 100] \times \text{scale ms}$	突触核设定
$\tau_{i,1:3}$	抑制性突触时间常数	$[75, 150, 300] \times \text{scale ms}$	慢抑制设定
scale	全局时间尺度	$\{0.75, 1.00, 1.25\}$	训练块内选择
ρ	同层复发强度	$\{0, 0.1, 0.2\}$	训练块内选择
λ	观测矩阵岭参数	$\{0.1, 0.3, 1, 3, 10, 30, 100\}$	训练块内选择

图 4.8 给出 A1 的同样本描述拟合和五个外层留出块的等权汇总。实线为观测 ERP，虚线为模型预测。上排用于观察模型在训练信息充分时可表示的波形形态，下排用于评价模型的外推能力。结果显示，模型能够生成与视觉提示同步、并在三通道呈现不同符号和尺度的预测；对于 F3、F4 中幅值较大的慢变成分，模型仍有进一步细化空间，因此后续重点结合留出结果评价其解释能力。

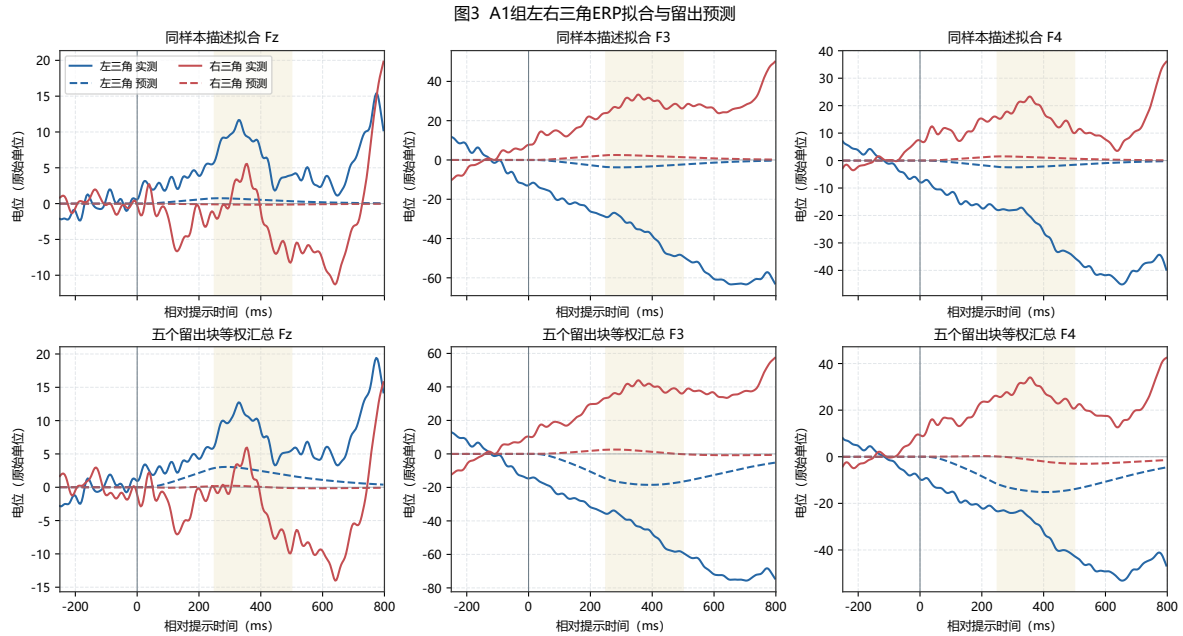


图 4.8: A1 左右条件 ERP 的同样本描述拟合与连续时间块留出预测

4.2.3 脑电信号差异的模型解释

(1) 左右差异的形成路径

在模型中，左右刺激之间的差异沿如下路径形成：镜像轮廓首先改变式 (2-1) 中的形状偏好输入；该差异经式 (2-2) 的 LGN 中继进入三级 E/I 网络，在不同时间常数作用下形成逐层延迟的源活动；最后由同一个 $\hat{\mathbf{L}}$ 投影为三通道电位。由于动力学参数和观测矩阵对左右条件共享，模型不允许通过“给左、右条件分别拟合一套参数”来制造差异。

为检验预测差异是否优于“左右完全相同”的平凡模型，定义右减左差异波

$$\mathbf{D}^{\text{obs}}(t) = \mathbf{y}_R(t) - \mathbf{y}_L(t), \quad \mathbf{D}^{\text{pred}}(t) = \hat{\mathbf{y}}_R(t) - \hat{\mathbf{y}}_L(t),$$

并构造差分预测增益

$$S_{\Delta} = 1 - \frac{\text{MSE}(\mathbf{D}^{\text{obs}}, \mathbf{D}^{\text{pred}})}{\text{MSE}(\mathbf{D}^{\text{obs}}, \mathbf{0})}. \quad (2-11)$$

当 $S_{\Delta} > 0$ 时，模型在该记录上的差异预测优于零差异基准； $S_{\Delta} < 0$ 表明当前参数配置相对零差异基准的增益较为有限。该指标是相对于零差异基准的预测增益，不等同于常规定义的决定系数 R^2 。

(2) 独立留出差异波结果

四组记录的留出差异波如图 4.9 所示。蓝色实线为实测右减左差异，红色虚线为模型预测，浅蓝阴影为块重采样得到的点态区间。A2、B2 在部分分析窗内能够跟踪差异波的方向和缓慢变化；A1、B1 的预测在部分时段与观测幅值存在偏差。随时间增大的区间宽度进一步反映了慢漂移和试次间波动对差异波估计的影响。

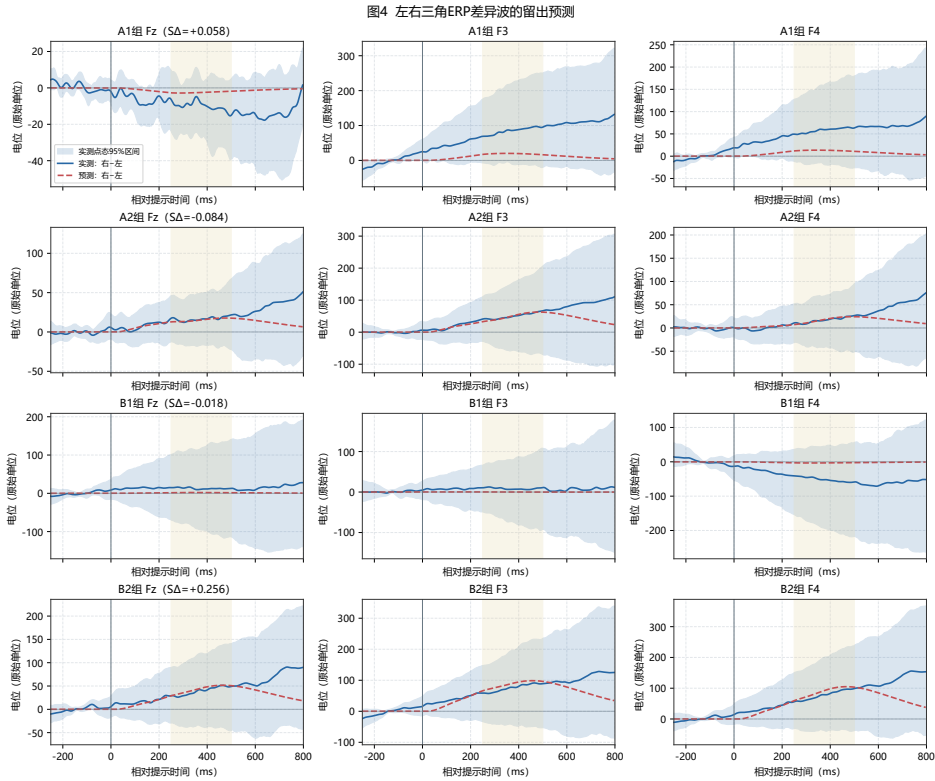


图 4.9: 四组记录右减左 ERP 差异波的连续时间块留出预测

表 6 给出各记录的定量结果。A1、B2 的 S_{Δ} 点估计为正，表明模型在这两组记录上捕捉到了优于零差异基准的方向变化；A2、B1 的点估计相对接近基准，且块重采样区间整体较宽。综合来看，

该模型已呈现一定的差异波外推能力，其跨记录稳定性仍有进一步提升空间。

表 6: 完整模型在四组记录上的留出预测结果

记录	S_{Δ}	S_{Δ} 块重采样区间	差异 RMSE	总 ERP R^2	波形相关系数
A1	0.0581	$[-0.4743, 0.1474]$	91.1154	-0.1825	-0.5342
A2	-0.0842	$[-1.0452, 0.4769]$	87.6182	0.1283	0.4166
B1	-0.0176	$[-0.0699, -0.0004]$	98.3928	-0.1027	-0.1545
B2	0.2562	$[-1.1102, 0.5607]$	90.6075	-0.1176	0.1193

(3) 额区空间侧化的解释

为描述双侧额区相对变化，在 250–500 ms 时窗内记三通道平均幅值为 A_{Fz}, A_{F3}, A_{F4} ，定义稳定化侧化指数

$$LI = \frac{A_{F4} - A_{F3}}{\max(|A_{F4}| + |A_{F3}|, \epsilon_{\text{train}})}. \quad (2-12)$$

其中， ϵ_{train} 由训练块幅值尺度的低分位数确定，用于防止分母接近零。图 4.10 的左、中列分别给出 F4–F3 电位差和 LI 的实测/预测曲线，右列给出分析窗内单试次 LI 分布。不同记录中的极性与幅值呈现一定个体差异，因此额区侧化更适合作为候选空间特征；其生理含义还需结合眼动、任务准备、参考电极及个体体传导差异进行综合分析。

图5 额区侧化特征的实测与预测

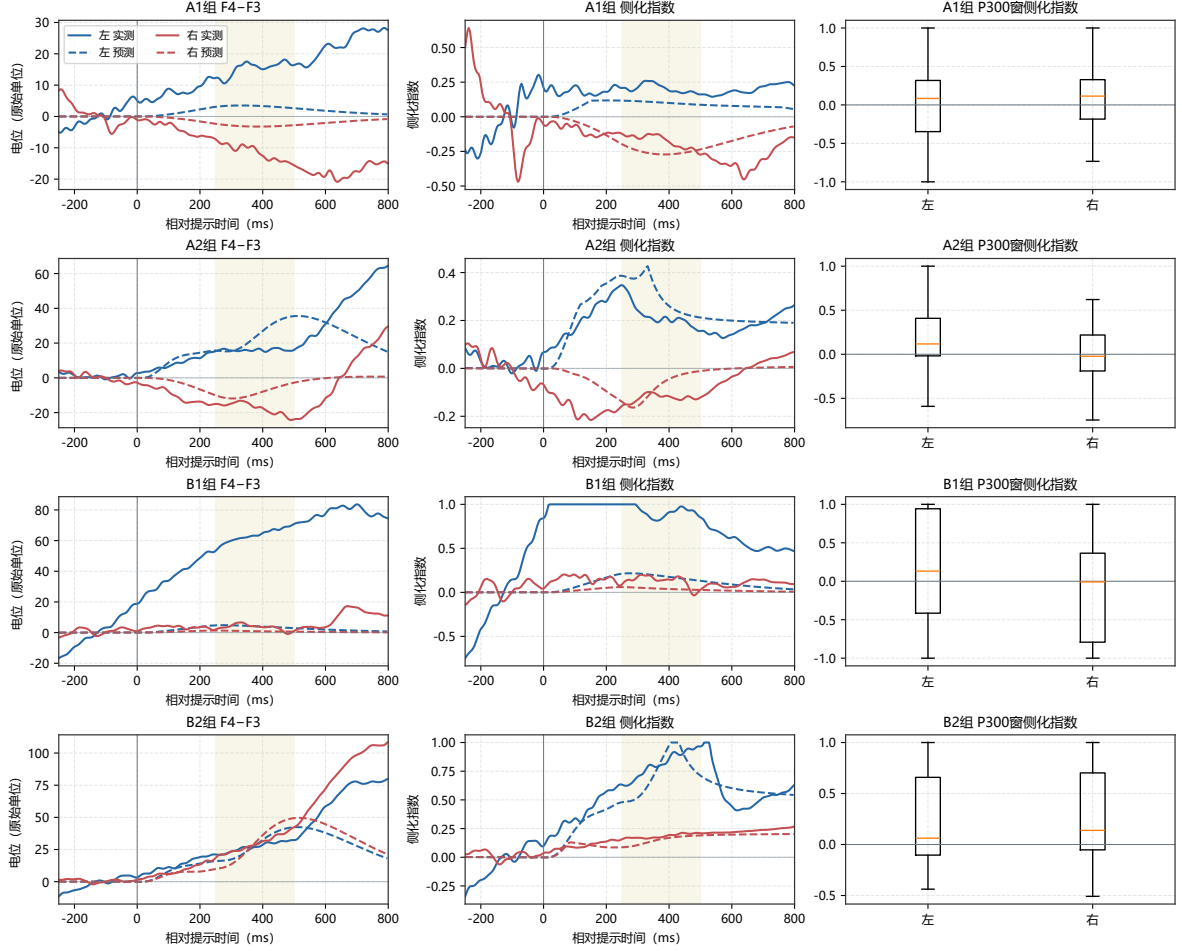


图 4.10: 四组记录的 F4-F3 电位差与稳定化侧化指数

(4) 消融与敏感性分析

为判定预测结果依赖模型中的哪些结构，分别进行三类反事实消融：令 $f_L = f_R$ 以取消形状选择性；令 $\rho = 0$ 以取消同层跨群复发；约束 F3、F4 观测行对称以取消观测侧化。同时，将时间尺度、复发强度及 F4 读出权重在固定范围内扰动，考察 S_Δ 的变化。

图 4.11 左上给出完整模型和多种替代模型的留出增益，右上为三项消融后的误差变化，左下为参数敏感性，右下为完整模型的块重采样区间。取消形状后模型的方向输出按结构归零，表明“形状选择性输入”是模型内部产生左右差异的必要条件；复发和对称观测消融在不同记录上呈现不同影响，反映了各结构对个体记录的贡献存在差异。A2、B2 的区间相对较宽，后续可通过增加重复记录进一步提高消融效应估计的精度。

图6 模型消融与参数敏感性

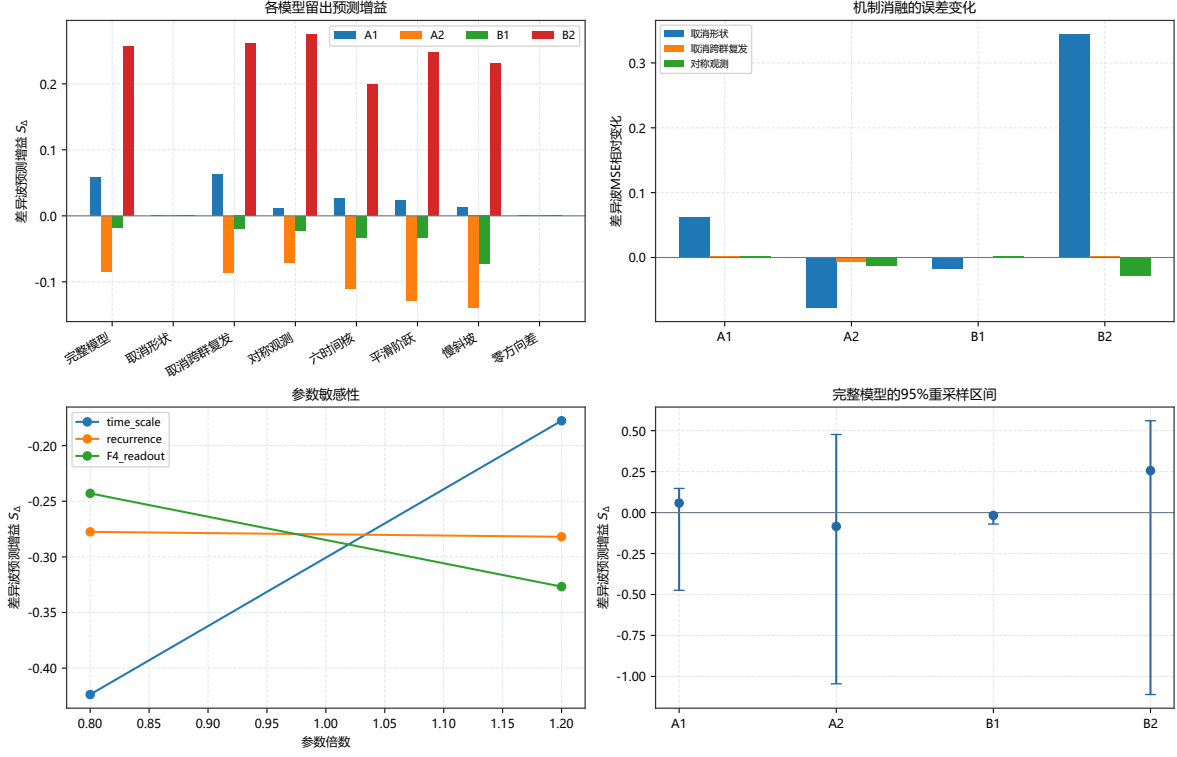


图 4.11: 模型消融、固定参数干预及差异预测增益的敏感性分析

(5) 模型解释的证据边界

综上，本模型形成了三层递进结果。第一，形状编码和级联动力学在数学上给出一条可计算的差异形成路径；第二，A1、B2 的正 S_{Δ} 表明该路径在部分记录上具有超越零差异基准的留出预测价值；第三，A2、B1 的点估计及较宽区间提示模型在跨记录适配方面仍有优化空间。考虑到现有观测通道数量和辅助信息条件，本文将 LGN、视觉层和额区层作为功能性计算模块，用于提供可解释的正向建模框架；更精细的解剖定位可在后续结合个体头模型、眼动与多通道数据进一步验证。

4.2.4 视觉刺激响应的特征表示

(1) 二十三维机制增强特征

为将正向模型转化为可供未知试次使用的表示，在每个试次上构造

$$\phi = [\mathbf{A}_{1:3}, \mathbf{P}_{1:3}, \mathbf{T}_{1:3}, \mathbf{M}_{1:3}, \text{LI}, A_{F4} - A_{F3}, \mathbf{b}_{1:6}, \mathbf{e}_{1:3}]^T \in \mathbb{R}^{23}. \quad (2-13)$$

各分量的定义见表 7。其中， $\mathbf{b}$ 为单试次在训练块共同模板和差异模板上的最小二乘投影系数， $\mathbf{e}$ 为相应的通道重构残差。构造测试试次特征时只使用训练块模板，不读取该试次的真实方向，因此该表示可用于严格的未知方向预测。

表 7: 二十三视觉刺激响应特征的组成

特征组	维数	计算区间或定义	表征含义
$\mathbf{A}_{1:3}$	3	250–500 ms 三通道有符号均值	中期响应幅值
$\mathbf{P}_{1:3}$	3	窗内满足阈值的局部正峰	峰值强度
$\mathbf{T}_{1:3}$	3	有效正峰的潜伏期	反应时序
$\mathbf{M}_{1:3}$	3	无合格峰时取 1	峰缺失状态
LI	1	式 (2-12)	稳定化额区侧化
$A_{F4} - A_{F3}$	1	分析窗内 F4 与 F3 幅值差	双侧额区梯度
$\mathbf{b}_{1:6}$	6	训练块共同/差异模板投影	机制模板坐标
$\mathbf{e}_{1:3}$	3	50–750 ms 通道重构 RMS	模型未解释成分
合计	23	—	时域、空间与机制信息联合表示

(2) 判别器与评价指标

对每个特征组使用收缩协方差线性判别分析。训练块内估计类别均值 $\boldsymbol{\mu}_L, \boldsymbol{\mu}_R$ 和收缩协方差 $\hat{\boldsymbol{\Sigma}}_\gamma$, 对未知试次 $\boldsymbol{\phi}$ 的判别函数写为

$$h(\boldsymbol{\phi}) = (\boldsymbol{\mu}_R - \boldsymbol{\mu}_L)^T \hat{\boldsymbol{\Sigma}}_\gamma^{-1} \boldsymbol{\phi} - \frac{1}{2} \left(\boldsymbol{\mu}_R^T \hat{\boldsymbol{\Sigma}}_\gamma^{-1} \boldsymbol{\mu}_R - \boldsymbol{\mu}_L^T \hat{\boldsymbol{\Sigma}}_\gamma^{-1} \boldsymbol{\mu}_L \right). \quad (2-14)$$

以平衡准确率

$$\text{BA} = \frac{1}{2} \left(\frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TN}}{\text{TN} + \text{FP}} \right) \quad (2-15)$$

作为主指标, 并同时报告 AUC 与 F1。置信区间采用 1000 次连续块重采样; 原假设检验在每个时间块内部置换标签 1999 次, 以保留块内类别数和时间结构; 七组特征之间再使用 maxT 对多重比较进行校正。另以 199 次块内循环移位作为时间依赖稳健性检查。

(3) 留出判别结果

表 8 汇总全部 275 个试次的外层留出结果。幅值特征取得最高的观测 BA 0.5161, 其 95% 块重采样区间覆盖 0.5, maxT 校正后 $p = 0.7855$; 23 维机制增强特征的 BA 为 0.4471、AUC 为 0.4679, 校正后 $p = 0.9995$ 。峰潜伏期、空间侧化和协方差投影等特征的结果主要分布在机会水平附近, 表明在当前样本规模下, 单试次方向信息的可分程度相对有限。

表 8: 不同视觉刺激响应特征的连续时间块留出判别性能

特征组	BA	BA 块重采样区间	AUC	F1	maxT 校正 p 值
时域幅值 (amplitude)	0.5161	[0.4846, 0.5481]	0.5337	0.5056	0.7855
峰潜伏期 (peak_latency)	0.4690	[0.4200, 0.5159]	0.4663	0.4632	0.9960
空间侧化 (spatial)	0.4693	[0.4332, 0.5004]	0.4721	0.4710	0.9960
23 维机制特征 (mechanism)	0.4471	[0.4016, 0.4837]	0.4679	0.4370	0.9995
协方差特征 (covariance)	0.4692	[0.4221, 0.5208]	0.4770	0.4672	0.9960
六维投影 (projection6)	0.4840	[0.4521, 0.5205]	0.4896	—	0.9795
三维方向投影 (direction3)	0.4919	[0.4451, 0.5378]	0.4632	—	0.9535

图 4.12 以 A1 为例给出 PCA 投影、各特征组留出 BA、机制特征混淆矩阵以及置换零分布。PCA 图中的训练点存在局部聚集，测试点则保留一定重叠；观测 BA 的红线落在置换分布主体内，与校正 p 值一致。机制特征相对空间特征的 BA 差值为 -2.2 个百分点，块重采样区间为 $[-4.9, 0.3]$ 个百分点，说明其当前增量收益较为有限，后续可通过丰富观测通道和扩大训练样本进一步提升。

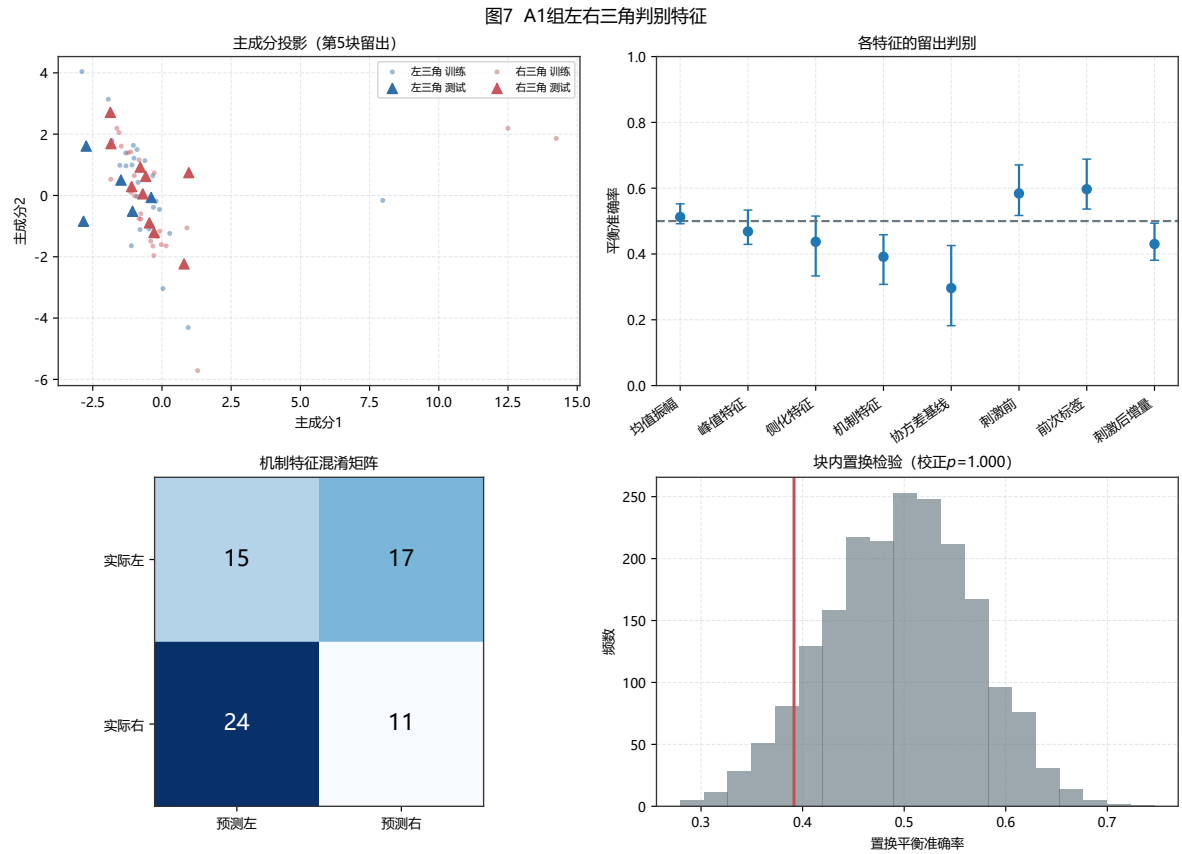


图 4.12: A1 视觉刺激特征投影、留出判别与块内置换检验

(4) 特征表示的使用结论

由上述结果可知，式 (2-13) 给出了一个计算闭合、对未知试次无标签泄漏的视觉刺激响应表示，可作为后续脑机接口模型的候选输入。现有两名受试者、四组记录和三个额区通道已呈现幅值、侧化与机制投影等多类候选差异，但单试次层面的区分度仍有提升空间。后续可通过增加受试者与重复记录，引入枕区/顶区电极和同步眼动通道，并在独立采集批次上固定时间窗、参数网格和最终判别器，以进一步提高特征表示的稳定性和迁移能力。

4.3 问题三模型的建立与求解

问题三要求在视觉提示、候选目标与应答行为之间建立连续的认知过程模型。为避免把应答后的动作电位混入认知状态，本文首先审计四组记录的事件语义，并按“应答前约 100 ms”原则构造变长脑电窗口；随后继承问题二的形状编码与视觉群体动力学，将提示保持、目标触发提取和认知整合表示为低维功能状态，再通过训练块内估计的有效观测矩阵映射至 F_z 、 F_3 、 F_4 ；最后以认知差分状态驱动随机证据累积过程，统一表示正确、错误和有限观察期内未决三类结局。真实 EEG 用于检验波形预测，仿真轨迹用于检验模型能否产生题目所述的异常应答情形，二者在解释上严格区分。

4.3.1 建模思路

(1) 从事件证据到认知状态的分层建模

本问采用“事件层—认知层—观测层—决策层”的递进结构。事件层确定提示、目标和应答的时间边界；认知层描述视觉输入在短时保持与目标触发下的状态演化；观测层把模型状态投影为头皮电位；决策层仅在仿真任务布局中把方向证据转换为行为结局。各层的输入、状态与输出如表 9 所示。该结构一方面复用问题二已建立的视觉通路，另一方面将无法由三导联 EEG 直接确认的深部来源保留为功能性状态，而不将其解释为海马体的直接源定位结果 [1,4]。

表 9: 问题三认知模型的分层结构

模型层次	主要输入	核心状态或运算	输出及作用
事件层	VisCue、TgtAct、记录边界	事件审计与应答前截窗	合格的变长认知脑电片段
视觉-认知层	提示形状、目标出现时刻	视觉输入 v 、记忆状态 m 、认知状态 p	连续认知源时间序列
头皮观测层	认知源与提示前背景	条件共享有效观测矩阵	三通道预测脑电 $\hat{x}$
行为决策层	方向对齐认知证据	有界随机证据累积	正确、错误与未决三类仿真结局
验证层	留出时间块与保存结果	误差比较、消融及敏感性分析	模型适用范围与证据边界

(2) 数据隔离与模型求解原则

每组 100 个提示事件按原始时间顺序划分为 5 个连续时间块。第 k 个时间块仅作为外层测试集，其余 4 个时间块用于选择时间常数、目标输入持续时间和岭惩罚强度，并估计头皮观测系数。任何由均值、尺度、背景斜率或候选参数产生的统计量均在训练块内部计算后作用于测试块。由此，模型接受的是一条未见过的原始认知试次，而不是已经利用该试次应答信息构造的条件平均波形。

模型求解依次完成四个步骤：首先确定每次试验可用的认知窗口；其次由问题二模型生成提示与目标的视觉驱动；再次联合求解低维认知状态和三通道观测映射；最后在固定认知状态上进行决策仿真、消融检验和观察窗敏感性分析。该路线对应“信号截取—状态建模—参数估计—外层验证”的闭环，能够分别回答认知信号如何截取、模型状态如何形成、头皮脑电如何观测以及异常应答如何处理。

(3) 候选模型与对照体系

为判断模型性能来自视觉瞬态、缓慢背景还是提示保持结构，本文不只拟合单一模型，而是建立由简到繁的候选族。各候选共享同一事件窗口、因果滤波、外层划分和评价指标，仅改变进入设计矩阵的状态列，因此模型间误差可以在相同测试试次上逐项比较。主模型记为 N2，其余候选分别承担零输入基准、视觉瞬态基准、单级认知基准和慢趋势对照等作用；同时通过删除记忆投影或关闭提取门，检验 N2 中各环节是否确有贡献。具体关系见表 10。

表 10: 问题三候选模型与对照关系

模型	保留的状态	主要用途	解释边界
B0	不含刺激状态，仅保留提示前背景外推	检查慢漂移可预测性	输入信息多于纯刺激模型
N0	提示与目标的视觉瞬态 v	视觉输入基准	不描述保持与提取
N1	v 与单级低通认知状态	检查一般时间平滑作用	不含独立记忆状态
N2	v 、 m 与 p	视觉-记忆-认知主模型	仅作功能性状态解释
N3	与事件对齐的平滑慢趋势基函数	排除一般慢变解释	不代表特定神经机制
N2 消融	删除记忆投影或关闭目标触发提取	定位主模型的结构贡献	与完整 N2 同一参数框架

其中，N2 与 N3 使用完全相同的外层测试块，二者的目标后误差差异用于回答“提示保持结构是否比一般慢趋势提供更多留出解释”；B0 则使用严格提示前 EEG 构造背景外推，属于更丰富的信息集合，只用于说明背景可预测性，不能与 N2 的机制贡献混为一谈。该对照体系把“能否拟合”“由何种信息拟合”以及“是否需要记忆结构”三个问题分开，避免仅凭复杂模型曲线更贴近实测波形便作出生理结论。

(4) 可观测性与因果边界

本问的直接观测量只有原始三通道 EEG、提示标记、候选目标标记及部分点击时刻； v 、 m 、 p 和决策变量 z 均为模型内部状态。建模时遵守三项边界：其一，神经状态只由提示、目标和此前样本驱动，应答时刻仅决定窗口终点，不反向进入状态方程；其二，头皮观测矩阵对左右提示共享，不能通过分别拟合两套系数人为制造方向差异；其三，决策层读取认知状态但不反馈修改 EEG，因而脑电预测与行为仿真可以分别审计。上述边界使每一项结果均能追溯至明确的信息来源，也为后续消融和独立留出检验提供可执行的判据。

4.3.2 视觉认知相关信号的截取

(1) 提示、目标与应答标记审计

设第 i 次试验的提示起点、候选目标起点、应答时刻、下一次提示时刻和记录终点分别为 t_i^c 、 t_i^T 、 t_i^r 、 t_i^n 与 t^{rec} 。通过对 VisCue 与 TgtAct 通道逐段扫描可知，四组记录均包含 100 个提示事件；A2、B2 的 ± 2 标记可作为独立点击，而 A1、B1 仅能以候选目标平台的结束时刻作为应答代理。由于数据未提供逐次正确目标映射和实验超时阈值，本文不依据点击方向推造正误标签，也不把缺少独立点击的项目一试验次直接记为超时。审计及质量控制结果见表 11。

表 11: 四组记录的事件标记与认知 EEG 保留情况

记录	提示事件数	独立点击数	平台末端代理数	无明确标记数	EEG 保留数
A1	100	0	100	0	65
A2	100	100	0	0	68
B1	100	0	100	0	64
B2	100	100	0	0	71
合计	400	200	200	0	268

(2) 应答前变长观察窗

为排除应答后的动作活动，令应答保护间隔 $\Delta = 0.1$ s，目标后行政观察上限 $D = 2$ s。对存在独立点击或平台末端代理的试次，认知窗口终点取应答前 100 ms、目标后上限、下一提示和记录终点中的最早者；对无明确应答标记但目标事件有效的试次，则去除应答项并保留有限观察窗。于是

$$T_i^{\text{end}} = \begin{cases} \min(t_i^r - \Delta, t_i^T + D, t_i^n, t^{\text{rec}}), & t_i^r \text{ 可用}, \\ \min(t_i^T + D, t_i^n, t^{\text{rec}}), & t_i^r \text{ 不可观测}. \end{cases} \quad (3-1)$$

式 (3-1) 中的 D 是统一的分析上限，而不是题目未提供的实验超时阈值。若点击晚于该上限，只在时间分析中视为行政右删失；若点击标记本身缺失，则标记为“应答不可观测”，可以进入 EEG 分析，但不进入真实点击的生存统计。正确、错误和正误未知的试次使用同一截窗规则，避免因行为结果不同而产生选择性别除。事件类型与截窗逻辑如图 4.13 所示。

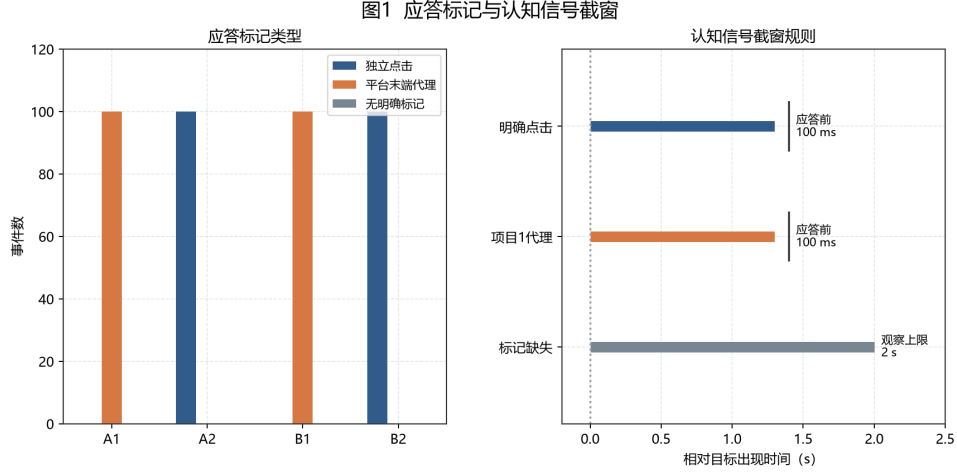


图 4.13: 四组记录的应答标记审计与认知信号截窗规则

(3) 因果预处理与质量控制

在每个连续时间块内，对原始 F_z 、 F_3 、 F_4 依次施加 60 Hz 陷波和 0.1–30 Hz 四阶 Butterworth 带通。与问题一用于离线 ERP 对齐的双向滤波不同，本问采用单向因果滤波，使时刻 t 的输出只依赖 t 及其之前的样本，从而避免未来应答附近信号反向泄漏至较早认知阶段。每个试次以提示前 $[-250, 0)$ ms 的中位数进行基线校正，并沿用硬件饱和、峰峰值和相邻采样跳变三类质量规则；阈值按变长窗口归一化后使用。

为比较不同阶段，定义编码、保持和提取三个不重叠时窗：

$$\mathcal{W}_i^E = [t_i^c, t_i^c + 0.8), \quad \mathcal{W}_i^M = [t_i^c + 0.8, t_i^T), \quad \mathcal{W}_i^R = [t_i^T, T_i^{\text{end}}]. \quad (3-2)$$

窗口外样本记为缺失，不通过时间伸缩把不同反应速度强行对齐。质量控制后 A1、A2、B1、B2 分别保留 65、68、64、71 个试次，共 268 个主 EEG 试次，为后续状态估计提供统一输入。

4.3.3 视觉--记忆--认知宏观模型与脑电差异解释

(1) 提示与目标的视觉驱动构造

问题二已给出左右三角形经形状编码、LGN 中继和三级 E/I 神经群传播后的末级视觉源。为避免将真实点击方向写入模型，本文只利用提示符号 $d_i \in \{-1, +1\}$ 构造提示的共同分量 $v_i^c(t)$ 与镜像差分分量 $v_i^A(t)$ ，并为候选目标设置与点击方向无关的共同输入 $v_i^T(t)$ ：

$$\mathbf{v}_i(t) = \begin{bmatrix} v_i^c(t) \\ d_i v_i^A(t) \\ v_i^T(t - t_i^T) \end{bmatrix}. \quad (3-3)$$

其中，前两项继承问题二形状选择性网络的共同/反对称输出，第三项由统一目标脉冲经相同视觉动力学产生。由此，目标出现时刻可以触发认知提取，但真实应答、点击侧和正误标签均不进入视觉源。

(2) 提示保持与目标触发提取

设 $\mathbf{m}_i(t) \in \mathbb{R}^2$ 为提示保持状态， $\mathbf{p}_i(t) = [p_i^c(t), p_i^A(t), p_i^T(t)]^T$ 为认知整合状态。提示信息以时间常数 τ_m 衰减，目标到达后通过门函数 $G_i^T(t) = 1(t \geq t_i^T)$ 触发延迟提取；独立目标通路不写入提

示记忆。其动力学方程为

$$\tau_m \dot{\mathbf{m}}_i(t) = -\mathbf{m}_i(t) + \begin{bmatrix} v_i^c(t) \\ d_i v_i^\Delta(t) \end{bmatrix}, \quad (3-4)$$

$$\begin{aligned} \tau_p \dot{p}_i^c(t) &= -p_i^c(t) + v_i^c(t) + G_i^T(t) m_i^c(t - \delta), \\ \tau_p \dot{p}_i^\Delta(t) &= -p_i^\Delta(t) + d_i v_i^\Delta(t) + G_i^T(t) m_i^\Delta(t - \delta), \quad \delta = 50 \text{ ms}, \\ \tau_p \dot{p}_i^T(t) &= -p_i^T(t) + v_i^T(t - t_i^T), \end{aligned} \quad (3-5)$$

式 (3-4)–(3-5) 将认知过程划分为“提示编码—缓慢保持—目标触发提取”三个环节。 $\mathbf{m}_i$ 表示功能性的保持变量， $\mathbf{p}_i$ 表示多通路整合后的宏观群体状态；二者可用于构造可检验的头皮电位预测，但在缺少个体 MRI、深部电极与高密度导联时，不把 $\mathbf{m}_i$ 唯一归因于海马体活动。

(3) 状态离散化与因果更新

模型按原始采样率 $f_s = 256 \text{ Hz}$ 在固定时间网格上积分，时间步长为 $\Delta t = 1/f_s$ 。对任一阶状态 $\tau_q \dot{\mathbf{q}} = -\mathbf{q} + \mathbf{u}$ ，采用指数 Euler 更新

$$\mathbf{q}[n] = a_q \mathbf{q}[n-1] + (1 - a_q) \mathbf{u}[n-1], \quad a_q = \exp\left(-\frac{\Delta t}{\tau_q}\right).$$

由于第 n 个状态只读取第 $n-1$ 个及更早输入，该离散形式保持因果性。50 ms 提取延迟约为 12.8 个采样间隔，计算时在记忆轨迹上进行线性分数延迟插值，而不简单取整为 13 个点，从而减少采样量化对目标触发时刻的影响。对每组记录，提示共同分量、提示差分分量和目标共同分量均先在统一时间轴上生成，再按该试次的 t_i^T 启动门函数；因此不同反应时只改变可评分终点，不改变目标前状态的计算结果。

从参数作用看， τ_m 决定提示信息跨越保持期的衰减速度， τ_p 决定目标后认知整合的响应速度， T_T 控制目标输入的持续范围， δ 则描述提取通路的固定传播延迟。四类参数分别影响波形的长期幅度、上升速度、驱动宽度和相位位置，具有相对清晰的结构分工；但在仅有三个额区通道时仍可能存在相关性，因此本文通过训练块内网格选择、岭惩罚和半合成参数恢复共同约束，而不把单次最小误差对应值直接视为生理真值。

(4) 头皮观测映射与参数估计

将式 (3-3)–(3-5) 的状态拼接为设计矩阵 $\mathbf{H}_i(t; \boldsymbol{\theta})$ ，经与实测 EEG 相同的因果滤波后，建立三通道有效观测模型

$$\hat{\mathbf{x}}_i(t) = \mathbf{H}_i(t; \boldsymbol{\theta}) \mathbf{B}, \quad \hat{\mathbf{x}}_i(t) \in \mathbb{R}^3, \quad (3-6)$$

其中， $\boldsymbol{\theta} = (\tau_m, \tau_p, T_T, \delta)$ 为动力学参数， $\mathbf{B}$ 为从视觉、记忆与认知状态到 F_z 、 F_3 、 F_4 的有效读出矩阵。同一外折内所有训练试次及左右提示共享 $\mathbf{B}$ ，只允许事件时刻和提示方向改变设计矩阵，因而测试波形不能依靠逐试次自由系数进行拟合。

考虑长时程脑电中还存在与任务无关的缓慢漂移，本文另设提示前背景辅助项。对每个通道，分别在三个提示前时间尺度上估计斜率 $s_{i,h,c}^{\text{pre}}$ ，并将其因果外推为

$$c_{i,h,c}(t) = s_{i,h,c}^{\text{pre}} \tau_h (1 - e^{-t/\tau_h}), \quad \tau_h \in \{0.25, 1, 4\} \text{ s}.$$

该项只使用提示出现前的样本，既不会读取测试试次未来脑电，也不改变式 (3-4)–(3-5) 的认知状态。纯刺激模型以 $\mathbf{H}_i$ 为输入；背景辅助设置则将 $\mathbf{c}_i$ 追加到设计矩阵后单独拟合和报告。两者信息集合不同，前者用于机制比较，后者用于评估实际长窗预测中背景条件的价值。

为使编码、保持和提取三个阶段在变长试次中具有相同权重，令 $w_i(t)$ 在每个非空阶段内归一化。训练块上的稳健估计写为

$$(\hat{\boldsymbol{\theta}}, \hat{\mathbf{B}}) = \arg \min_{\boldsymbol{\theta}, \mathbf{B}} \sum_{i \in \mathcal{T}} \sum_t w_i(t) \rho_{1.5} \left(\frac{\mathbf{x}_i(t) - \mathbf{H}_i(t; \boldsymbol{\theta}) \mathbf{B}}{s_{\mathcal{T}}} \right) + \lambda \|\mathbf{P} \mathbf{B}\|_F^2, \quad (3-7)$$

式中， $\rho_{1.5}(\cdot)$ 为逐通道作用的 Huber 损失， $\mathbf{s}_T$ 为训练块尺度， $\mathbf{P}$ 对方向相关状态施加较强收缩， λ 为岭惩罚系数。具体求解分为两个阶段：先对全部候选参数与 λ 组合计算训练块内嵌套岭回归误差，筛出排名最靠前的两个组合；再对这两个组合逐折执行 Huber 迭代加权最小二乘，并按平均稳健损失确定最终参数。最后仅用四个外层训练块重估 $\mathbf{B}$ ，直接预测未参与任何选择的第五块。该“全网格筛选—稳健复核—外层预测”策略兼顾计算效率与异常样本抵抗能力。主要候选范围如表 12 所示。

表 12: 认知模型与决策层的主要参数

参数	含义	候选值或设定	确定方式
τ_m	提示保持时间常数	$\{0.5, 1, 2, 4\}$ s	训练块内选择
τ_p	认知整合时间常数	$\{0.1, 0.3, 0.6\}$ s	训练块内选择
T_T	目标输入持续时间	$\{0.1, 0.2, 0.4\}$ s	训练块内选择
δ	提取延迟	0.05 s	固定设定
r_d	方向状态收缩倍数	$\{1, 10\}$	训练块内选择
λ	岭惩罚强度	$\{10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$	嵌套验证
γ	决策证据增益	1.5	参考仿真设定
σ	扩散噪声强度	0.6	参考仿真设定
b	决策边界	1	归一化设定

(5) 认知状态对脑电差异的解释

左右提示的差异首先表现为 $d_i v_i^\Delta$ 的符号反转，随后在 τ_m 控制下形成不同方向的保持轨迹；目标到达时，门函数把延迟记忆状态重新注入 p_i^Δ ，经同一个观测矩阵投影为三导联差异。模型不为左右条件分别拟合两套参数，因此差异必须由提示的镜像输入及其随时间传播产生。为检查慢变背景的影响，另构造只使用提示前 EEG 斜率的背景辅助观测项，并与纯刺激输入模型分别报告。

图 4.14 给出四组记录在 F_z 通道上的留出平均预测。蓝线为式 (3-6) 的刺激-认知模型，橙线在其信息集合之外加入严格提示前背景，黑线为实测脑电。背景辅助模型对长时间慢漂移的跟踪更充分，四组目标后均方误差相对纯刺激模型约下降 22%–56%；这一变化说明提示前背景对长窗预测具有价值，但不能据此把收益归因于记忆或海马机制。纯刺激模型则保留了更清晰的机制解释路径，用于后续记忆状态与慢趋势对照。

图2 认知模型预测与实测脑电

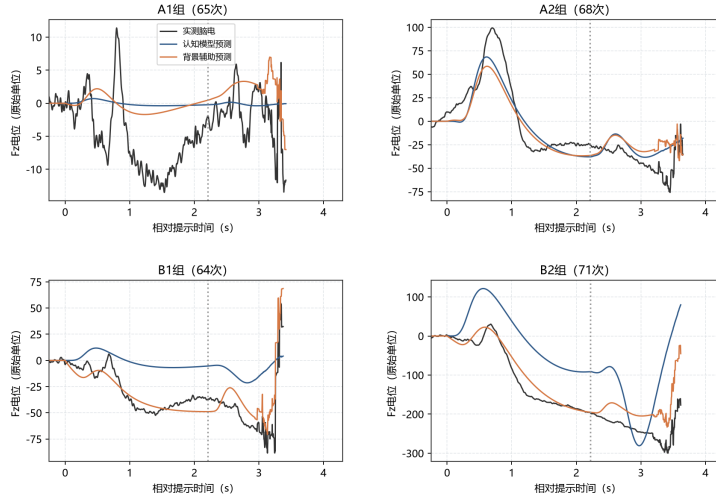


图 4.14: 四组记录的认知模型留出预测与实测 F_z 脑电对比

4.3.4 认知状态驱动三类应答模型

(1) 认知证据与目标匹配

取未经过头皮观测滤波的认知差分状态 $p_i^\Delta(t)$, 以参考状态最大幅度 a 归一化, 并按提示方向对齐:

$$e_i(t) = \frac{d_i p_i^\Delta(t)}{a}, \quad a = \max_t |p_{\text{ref}}^\Delta(t)|. \quad (3-8)$$

为使提示记忆能够与左右候选目标相比较, 进一步复用问题二的形状编码。设归一化提示特征为 $\mathbf{f}_i^c$, 模拟布局中左、右目标特征为 $\mathbf{f}_i^L$ 与 $\mathbf{f}_i^R$, 则方向匹配系数取为

$$r_i = \frac{\langle \mathbf{f}_i^c, \mathbf{f}_i^R \rangle - \langle \mathbf{f}_i^c, \mathbf{f}_i^L \rangle}{\kappa},$$

其中, κ 由左右标准形状的余弦距离归一化, 使交换左右目标位置时 r_i 符号反转。在主机制扰动实验中, 为单独考察记忆保持、证据增益与噪声的作用, 取 $r_i = s_i^*$; 在目标匹配扩展中则由上式计算 r_i 。正确目标侧 $s_i^* \in \{-1, +1\}$ 仅由平衡的模拟任务布局给定并用于结局评分, 不由真实 MAT 文件中的点击方向反推, 也不进入形状相似度计算。

决策变量 $z_i(t)$ 服从有界随机累积方程

$$dz_i(t) = r_i \gamma e_i(t) dt + \sigma dW_i(t), \quad z_i(0) = 0, \quad |z_i(t)| < b, \quad (3-9)$$

其中, γ 为证据增益, σ 为扩散噪声, $W_i(t)$ 为标准 Wiener 过程, b 为对称决策边界。 z_i 只读取认知证据, 不作为新的头皮源, 也不反馈修改 $\mathbf{m}_i$ 或 $\mathbf{p}_i$; 模型未单独估计感觉-运动转换和按键执行耗时, 因此首次越界时间表示模型决策时间, 而不直接等同于实测点击间隔。图 4.15 展示基准、短保持和关闭提取三种情况下的记忆状态、认知状态与决策驱动。缩短 τ_m 会使目标出现前的可用证据衰减; 关闭提取则使目标后差分状态接近零, 从机制上给出迟疑或错误应答的形成路径。

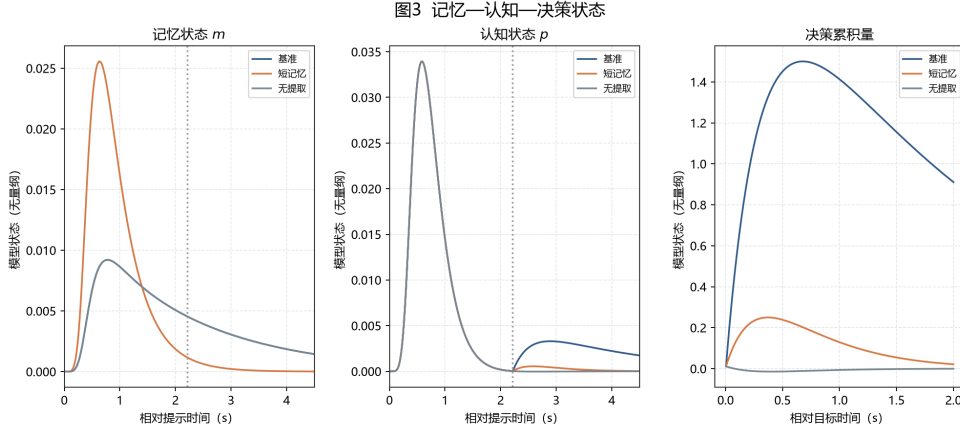


图 4.15: 提示保持、目标触发认知状态与决策证据的连续演化

(2) 正确、错误与未决结局的统一判定

令 $T_i = \inf\{t : |z_i(t)| \geq b\}$ 为首次越界时间，仿真截止时刻为 D 。若 $T_i \leq D$ ，则 $\text{sign}[z_i(T_i)]$ 给出选择；若截止前未越界，则该轨迹记为未决并在 D 处右删失。三类结果定义为

$$Y_i = \begin{cases} \text{正确,} & T_i \leq D, \text{sign}[z_i(T_i)] = s_i^*, \\ \text{错误,} & T_i \leq D, \text{sign}[z_i(T_i)] \neq s_i^*, \\ \text{未决,} & T_i > D. \end{cases} \quad (3-10)$$

在任意时刻 t ，分别记正确越界、错误越界的累计概率为 $F_c(t)$ 、 $F_e(t)$ ，未决概率为 $S_u(t)$ ，则模型逐时满足

$$F_c(t) + F_e(t) + S_u(t) = 1.$$

这一守恒关系保证错误轨迹不会被剔除，未决轨迹也不会被强制归入错误。数值实现采用 $\Delta t = 1/256$ s 的首次越界判定，一旦轨迹触及边界便停止更新其选择和决策时间；不同机制场景使用共同随机数，使比例变化主要反映被扰动参数而非蒙特卡洛样本差异。

未决轨迹只保存观察时长 D ，不把 D 或 0 伪装成反应时间。对实测项目二点击，采用同样的右删失定义计算任意点击的描述性生存曲线：若时刻 u_j 的风险集为 n_j 、点击数为 d_j ，则

$$\hat{S}(t) = \prod_{u_j \leq t} \left(1 - \frac{d_j}{n_j}\right). \quad (3-11)$$

该曲线反映“截至时刻 t 尚未出现点击”的比例，但不区分点击是否正确，也不把项目一平台末端代理当作真实点击事件。

(3) 机制扰动与三类决策仿真

在参考设置 $\tau_m = 2$ s、 $\tau_p = 0.3$ s、 $\gamma = 1.5$ 、 $\sigma = 0.6$ 、 $b = 1$ 、 $D = 2$ s 下，对每个场景生成 6000 条轨迹，并使用相同随机数实施单因素比较。表 13 与图 4.16 给出结果。基准场景的证据持续且噪声适中，轨迹大多在正确边界越界；短保持、关闭提取和低证据增益均提高未决比例；高噪声主要增加错误越界；缩短截止时间则在基本不改变早期错误率的同时增加行政未决比例。上述结果说明同一认知状态模型能够生成题目要求考虑的多种应答情形。

表 13: 不同机制扰动下的三类决策仿真结果

场景	轨迹数	正确比例	错误比例	未决比例	受限平均时间/s
基准	6000	0.9742	0.0010	0.0248	0.8665
短保持	6000	0.3315	0.1405	0.5280	1.6000
关闭提取	6000	0.2150	0.2310	0.5540	1.6305
高噪声	6000	0.7737	0.2128	0.0135	0.5612
低证据增益	6000	0.3072	0.1563	0.5365	1.6161
短仿真截止	6000	0.8300	0.0010	0.1690	0.8095

图4 认知模型的三类决策结果

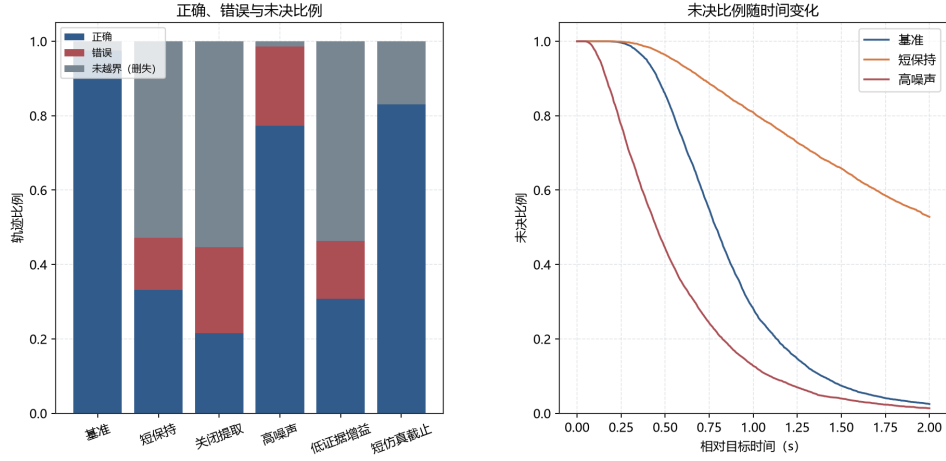


图 4.16: 认知状态驱动的正确、错误与未决决策仿真

表 13 中的比例是给定模拟目标布局 and 参数下的模型输出，Wilson 区间只反映蒙特卡洛抽样误差，不是受试者总体置信区间。由于原始数据未提供正确目标映射和正式超时规则，本文不将其解释为两名受试者的实测正确率或超时率；其作用是验证模型结构能够以统一方程表示正确、错误和未决三种可能结局。

4.3.5 模型求解结果与验证

(1) 连续时间块外层留出验证

对每个外层测试块，使用训练块确定的参数和读出矩阵重建三通道 EEG。令 $\mathcal{I}_{g,c}$ 表示阶段 g 、通道 c 的有效测试样本集合，则阶段均方误差为

$$\text{MSE}_g = \frac{1}{3} \sum_{c=1}^3 \frac{1}{|\mathcal{I}_{g,c}|} \sum_{(i,t) \in \mathcal{I}_{g,c}} [x_{i,c}(t) - \hat{x}_{i,c}(t)]^2. \quad (3-12)$$

为判断目标后记忆-认知结构是否优于一般慢变化，设置不含提示记忆、仅使用平滑慢趋势基函数的 N3 对照，并定义相对预测增益

$$S_g = 1 - \frac{\text{MSE}_g^{\text{N2}}}{\text{MSE}_g^{\text{N3}}}. \quad (3-13)$$

$S_g > 0$ 表示认知模型在该阶段的留出误差更低。表 14 给出目标后提取阶段的结果，区间由 2000 次连续时间块重采样获得。A1、A2、B1 的点估计为正，B2 略为负；四组区间均跨越 0，说明现有三导联、两名受试者记录支持将该结构作为可计算的候选认知模型，但尚不足以把缓慢成分唯一解释为特定深部记忆源。

表 14: 认知模型相对慢趋势对照的目标后留出预测增益

记录	EEG 试次	S_R	95% 块重采样区间	结果方向
A1	65	0.0244	$[-0.0134, 0.0742]$	正向
A2	68	0.0022	$[-0.0125, 0.0114]$	正向
B1	64	0.0034	$[-0.0028, 0.0163]$	正向
B2	71	-0.0148	$[-0.0547, 0.0298]$	轻微负向

(2) 观察窗与点击时间敏感性

将目标后行政观察上限分别设为 1.5、2.0 与 2.5 s，使用同一组冻结参数，并仅在各上限共同保留的试次上重新评分。由图 4.17 左侧可知，四组 S_R 的点估计随观察上限基本稳定，结论不依赖单一 2 s 设置；右侧给出 A2、B2 的实测任意点击时间，两组曲线均在约 1.0 s 后开始下降，并在 2 s 附近接近零。该结果只用于说明窗口覆盖了现有点击时间分布，点击间隔同时包含认知与运动过程，不能直接等同于模型决策时间。

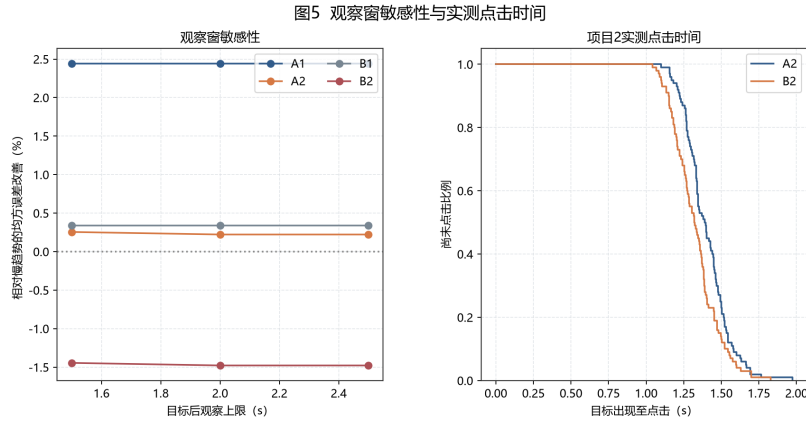


图 4.17: 目标后观察窗敏感性与项目二实测点击时间分布

(3) 消融、参数恢复与数值审计

除慢趋势对照外，本文分别关闭记忆向认知状态的投影、关闭目标触发提取，并改变 τ_m 、 γ 、 σ 和 D ，检查结论是否由单一环节偶然造成。进一步将记忆时间常数搜索范围扩展至 0.25–8 s，新拟合相对原参数网格未呈现稳定的外层误差改善，因而保留训练内选择的主结果。半合成参数恢复表明：无噪声时 9 个场景均能恢复生成时间常数；噪声为 0.5 倍时精确恢复率为 0.333，噪声为 1 倍时精确恢复率下降至 0。该结果说明求解程序在理想条件下可辨识，但真实噪声会显著增加时间常数的不确定性，故本文不对单个受试者报告唯一生理参数。

最终验证共通过 28 项自动测试，覆盖未来 EEG 不影响过去状态、外层测试块不参与参数估计、目标不写入提示记忆、缺失点击不等于超时、平台代理不进入真实生存曲线以及三类结局概率守恒等关键性质。独立审计重新计算 31356 条阶段误差记录，最大相对差异为 3.17×10^{-16} ；冻结预测重

建的最大绝对差异为 0；36000 条决策轨迹在时间步长减半后，三类比例的最大绝对变化为 0.00375。上述结果验证了程序实现、数据隔离和数值计算的一致性。

综上，本文构建了由视觉编码、提示保持、目标触发提取、头皮观测与随机决策组成的认知宏观模型。该模型能够在同一框架下处理应答代理、独立点击和潜在的未决试次，并在连续时间块留出条件下生成可与实测 EEG 比较的预测波形。现有结果支持其作为视觉认知过程的功能性解释与仿真平台；对深部脑区来源、真实正误及实验超时的进一步确认，仍需结合逐试次目标布局、高密度 EEG 或独立深部观测数据。

五、 模型优缺点评价

5.1 问题一模型评价

模型的优点：该模型以垂直/水平分量、软门控和均值保护兼顾伪影抑制与响应保留，并通过五折留出和训练内半合成选型减少信息泄漏。四组代理 MAE 降幅为 5.4%–10.0%，左右差异保留率为 0.813–0.952；三次 B 样条可进一步给出连续响应曲线。

模型的缺点：数据缺少同步眼电、眼动和独立无噪声脑电，代理误差下降不能直接等同于真实神经信号恢复。四组 SNR 代理增量区间均跨越零点，弱扰动与部分时间分段下的收益仍有波动，现有结果主要适用于已知条件下的离线响应描述。

改进思路：增加眼电、眼动及中央、顶区、枕区电极，在独立受试者上固定参数验证；依据训练块信噪比自适应调节收缩强度，并用时延校准的因果滤波替代双向滤波。

5.2 问题二模型评价

模型的优点：模型构建了“形状编码–LGN 中继–三级 E/I 群体–突触源–头皮观测”的完整计算链，左右条件共享参数，并用消融检验结构贡献。连续时间块留出、1999 次块内置换和 maxT 校正将机制解释、差异预测与单试次判别分开评价，二十三特征亦可用于未知试次。

模型的缺点：仅有 Fz、F3、F4 三个额区电极，缺少枕顶区、电眼和个体头模型，有效观测矩阵不能作为唯一神经源定位。简单幅值和机制特征的总体 BA 分别为 0.5161、0.4471，跨块方向信息较弱，稳定判别能力仍有提升空间。

改进思路：增加高密度 EEG、同步眼动和个体 MRI，将共同背景与方向差异分别正则化；固定时间窗与候选特征，并在扩大受试者规模后检验跨个体稳定性。

5.3 问题三模型评价

模型的优点：模型区分独立点击、平台代理和应答不可观测三类事件，按应答前约 100 ms 截取信号；并将视觉编码、提示保持、目标提取、头皮观测与随机决策统一起来，表示正确、错误和未决三类结局。时间块留出、窗口敏感性、消融和数值审计保证了过程可复核。

模型的缺点：数据缺少逐试次正确目标、完整布局和正式超时阈值，三类决策比例只能作为机制仿真结果。三导联 EEG 不能直接定位海马体等深部来源；相对慢趋势对照的四组区间均覆盖零点，且时间常数恢复精度随噪声增加而下降。

改进思路：完整记录目标布局、正误、超时和动作触发信息，增加高密度 EEG、眼电与独立深部观测；采用分层状态空间模型区分认知决策和运动时间，并在新增受试者上检验泛化能力。

参考文献

- [1] GLOMB K, CABRAL J, CATTANI A, et al. Computational models in electroencephalography[J]. Brain Topography, 2022, 35(2): 142–161.
- [2] WILSON H R, COWAN J D. Excitatory and inhibitory interactions in localized populations of model neurons[J]. Biophysical Journal, 1972, 12(1): 1–24.
- [3] STEEGHS-TURCHINA M, SRINIVASAN R, NUNEZ P L, et al. Slow wave dynamics of scalp EEG can be explained by simple statistical models of long-range connections[J]. NeuroImage, 2025, 321: 121418.
- [4] DAUME J, KAMINSKI J, et al. Control of working memory by phase-amplitude coupling of human hippocampal neurons[J]. Nature, 2024, 629: 808–816.
- [5] ACEBRÓN J A, BONILLA L L, PÉREZ VICENTE C J, et al. The Kuramoto model: A simple paradigm for synchronization phenomena[J]. Reviews of Modern Physics, 2005, 77(1): 137–185.
- [6] AZZOPARDI G, PETKOV N. Ventral-stream-like shape representation: from pixel intensity values to trainable object-selective COSFIRE models[J]. Frontiers in Computational Neuroscience, 2014, 8: 112.

附录

问题一建模代码	编程语言：Python
<pre> # 问题一：脑电预处理、双分量校正与样条拟合 import numpy as np import pandas as pd from scipy.interpolate import LSQUnivariateSpline import scipy.io as sio from scipy.stats import median_abs_deviation from scipy.signal import find_peaks, butter, sosfiltfilt, iirnotch, filtfilt, savgol_filter, welch FS_EXPECTED = 256 PRE_SEC = 0.25 # 刺激前 250 ms 基线 POST_SEC = 0.80 # 刺激后 800 ms 分析窗 N_PRE = round(PRE_SEC * FS_EXPECTED) N_POST = round(POST_SEC * FS_EXPECTED) TIMES = np.arange(-N_PRE, N_POST) / FS_EXPECTED TIMES_MS = TIMES * 1000.0 P300_MASK = (TIMES >= 0.25) & (TIMES <= 0.50) CHANNEL_NAMES = ["Fz", "F3", "F4"] # 双分量校正参数 CORRECTION_PARAMS = { "win_blink": 41, # 垂直眼电平滑窗长 (约 160 ms) "win_saccade": 31, # 水平扫视平滑窗长 (约 120 ms) "gate_z_v": 2.2, # 垂直软门控阈值 "gate_z_h": 2.2, # 水平软门控阈值 "gate_width": 1.2, # 软门控过渡带宽度 "reg_lambda": 0.05, # 岭回归正则化系数，防止过减 "max_beta_v": 1.8, # 垂直通道最大增益 "max_beta_h": 1.5, # 水平通道最大增益 } def load_mat_file(filepath): """读取赛题 .mat 文件""" mat = sio.loadmat(filepath, squeeze_me=True, struct_as_record=False) fs = int(mat["SampleRate"]) labels = list(mat["DataLabel"]) data = mat["data"] return fs, labels, data def extract_viscue_events(viscue): """根据 VisCue 通道提取事件上升沿与方向标签 (-1: 左三角, +1: 右三角)""" nonzero = (viscue != 0) onset_mask = nonzero & np.r_[True, ~nonzero[:-1]] onsets = np.where(onset_mask)[0] cue_values = viscue[onsets].astype(int) return onsets, cue_values def segment_epochs(signal, onsets, n_pre=N_PRE, n_post=N_POST): """截取 [-N_PRE, N_POST] 试次片段, shape: (trials, channels, time)""" epochs = [] n_samples = signal.shape[-1] for onset in onsets: start = onset - n_pre end = onset + n_post </pre>	

```

        if start < 0 or end > n_samples:
            continue
        epochs.append(signal[..., start:end])
    return np.stack(epochs, axis=0)

def preprocess_continuous_eeg(data_3ch, fs=FS_EXPECTED):
    """
    对连续 3 通道 EEG 进行预处理：
    1. 60 Hz 陷波滤波（抑制实测强工频干扰）
    2. 0.1 - 30 Hz 四阶 Butterworth 零相位带通
    """
    # 60 Hz 陷波
    b_notch, a_notch = iirnotch(w0=60.0, Q=30.0, fs=fs)
    notched = filtfilt(b_notch, a_notch, data_3ch, axis=-1)

    # 0.1-30 Hz 带通（4阶 Butterworth, 前后双向零相位）
    sos = butter(4, [0.1, 30.0], btype="bandpass", fs=fs, output="sos")
    filtered = sosfiltfilt(sos, notched, axis=-1)
    return filtered

def robust_baseline_correct(epochs, n_pre=N_PRE):
    """
    使用刺激前 [-250, 0] ms 的稳健中位数/截断均值作为基线，扣除慢漂移偏置
    """
    baseline = np.median(epochs[:, :, :n_pre], axis=2, keepdims=True)
    return epochs - baseline

def robust_zscore(x):
    """基于 Median 与 MAD 的稳健 Z-score 标准化"""
    med = np.median(x)
    mad = median_abs_deviation(x, scale="normal")
    if mad < 1e-12:
        mad = np.std(x) + 1e-12
    return (x - med) / mad

def detect_bad_trials(raw_epochs, filtered_epochs):
    """
    识别不可恢复严重坏试次：
    1. 原始采样点发生 ±1000 硬件饱和截幅连续超过 15 个点
    2. 峰峰值超过 1800 原始幅值单位
    3. 相邻点发生物理不可能的跳跃
    """
    n_trials = len(raw_epochs)
    peak_to_peak = np.ptp(filtered_epochs, axis=2).max(axis=1)

    # 硬件饱和检测 (|raw| >= 999.0)
    sat_points = np.sum(np.abs(raw_epochs) >= 999.0, axis=(1, 2))

    # 最大跳跃与最大绝对值
    max_jump = np.max(np.abs(np.diff(filtered_epochs, axis=2))), axis=(1, 2))
    max_abs = np.max(np.abs(filtered_epochs), axis=(1, 2))

    # 综合伪影得分

```

```

z_ptp = robust_zscore(np.log1p(peak_to_peak))
z_jump = robust_zscore(np.log1p(max_jump))
z_abs = robust_zscore(np.log1p(max_abs))
artifact_score = np.maximum(z_ptp, 0) + np.maximum(z_jump, 0) + np.maximum(z_abs, 0)

irrecoverable = (sat_points >= 15) | (peak_to_peak > 1800) | (max_jump > 600)
return irrecoverable, artifact_score, peak_to_peak

def build_clean_reference_model(epochs, cues, irrecoverable, artifact_scores, ref_ratio=0.50):
    """
    在训练试次中为左(-1)与右(+1)刺激分别构建低伪影 ERP 模板及动态残差尺度
    """
    templates = {}
    scales_v = {}
    scales_h = {}
    ref_mask = np.zeros(len(epochs), dtype=bool)

    for cond in (-1, 1):
        valid_idx = np.where((cues == cond) & (~irrecoverable))[0]
        if len(valid_idx) == 0:
            continue
        # 按伪影得分排序, 选取最干净的前 ref_ratio 试次
        sorted_idx = valid_idx[np.argsort(artifact_scores[valid_idx])]
        k = max(8, int(np.ceil(ref_ratio * len(sorted_idx))))
        chosen = sorted_idx[:k]
        ref_mask[chosen] = True

        # 稳健中位数模板 (shape: 3, time)
        tpl = np.median(epochs[chosen], axis=0)
        templates[cond] = tpl

        # 计算纯净试次的残差与伪影参考
        res = epochs[chosen] - tpl[None, :, :]
        # 垂直分量: 三通道中位数
        res_v = np.median(res, axis=1)
        # 水平分量: (F4 - F3) 偶极差分
        res_h = 0.5 * (res[:, 2, :] - res[:, 1, :])

        scale_v = median_abs_deviation(res_v, axis=0, scale="normal")
        scale_h = median_abs_deviation(res_h, axis=0, scale="normal")

        scale_v = np.maximum(scale_v, 0.5 * np.median(scale_v) + 1e-6)
        scale_h = np.maximum(scale_h, 0.5 * np.median(scale_h) + 1e-6)

        scales_v[cond] = scale_v
        scales_h[cond] = scale_h

    return templates, scales_v, scales_h, ref_mask

def correct_single_epoch(epoch, template, scale_v, scale_h, params=CORRECTION_PARAMS):
    """
    双分量自适应校正:
    1. 从单试次中扣除对应的先验 ERP 模板, 得到纯残差 residual
    """

```

```

2. 分离垂直眼电分量 V(t) 与水平扫视分量 H(t)
3. 分别进行多尺度平滑与动态自适应软门控
4. 采用带约束岭回归自适应估计各通道的消除系数
5. 重构无伪影信号并恢复基线
"""

residual = epoch - template # shape: (3, time)

# 1. 垂直同相分量参考 (眨眼/头动)
ref_v = np.median(residual, axis=0)
ref_v_smooth = savgol_filter(ref_v, window_length=params["win_blink"], polyorder=3)

# 2. 水平反相偶极分量参考 (左右扫视 eye movement)
ref_h = 0.5 * (residual[2] - residual[1])
ref_h_smooth = savgol_filter(ref_h, window_length=params["win_saccade"], polyorder=3)

# 3. 动态软门控
z_v = np.abs(ref_v_smooth) / (scale_v + 1e-6)
gate_v = np.clip((z_v - params["gate_z_v"]) / params["gate_width"], 0, 1)
gate_v = savgol_filter(gate_v, window_length=15, polyorder=2)
gate_v = np.clip(gate_v, 0, 1)
art_v = ref_v_smooth * gate_v

z_h = np.abs(ref_h_smooth) / (scale_h + 1e-6)
gate_h = np.clip((z_h - params["gate_z_h"]) / params["gate_width"], 0, 1)
gate_h = savgol_filter(gate_h, window_length=15, polyorder=2)
gate_h = np.clip(gate_h, 0, 1)
art_h = ref_h_smooth * gate_h

corrected = epoch.copy()

# 4. 通道自适应岭回归消除
X = np.column_stack([art_v, art_h]) # shape: (T, 2)
lambda_eye = params["reg_lambda"] * np.eye(2)
XtX = X.T @ X + lambda_eye

for ch in range(3):
    y = residual[ch]
    # 解岭回归系数: beta = (X^T X + lambda I)^(-1) X^T y
    beta = np.linalg.solve(XtX, X.T @ y)

    # 垂直分量在额叶均为正偏转
    beta_v = float(np.clip(beta[0], 0, params["max_beta_v"]))
    # 水平分量允许正负 (F3 为负, F4 为正)
    beta_h = float(np.clip(beta[1], -params["max_beta_h"], params["max_beta_h"]))

    # 若是 Fz (中线), 水平扫视分量理论为 0, 抑制 beta_h
    if ch == 0:
        beta_h = float(np.clip(beta_h, -0.3, 0.3))

    corrected[ch] = epoch[ch] - (beta_v * art_v + beta_h * art_h)

# 5. 重新校正刺激前基线
corrected -= np.mean(corrected[:, :N_PRE], axis=1, keepdims=True)

```

```

    return corrected

def compute_bootstrap_ci(epochs_matrix, n_boot=500, ci=95):
    """
    对试次集合进行 Bootstrap 重采样，计算逐时间点的 95% 置信区间
    shape: (trials, time) -> (time,), (time,)
    """
    n_trials = len(epochs_matrix)
    if n_trials < 5:
        mean_sig = np.mean(epochs_matrix, axis=0)
        return mean_sig, mean_sig

    rng = np.random.default_rng(42)
    boot_means = np.zeros((n_boot, epochs_matrix.shape[1]))
    for b in range(n_boot):
        sample_idx = rng.choice(n_trials, size=n_trials, replace=True)
        boot_means[b] = np.mean(epochs_matrix[sample_idx], axis=0)

    alpha = (100 - ci) / 2.0
    lower = np.percentile(boot_means, alpha, axis=0)
    upper = np.percentile(boot_means, 100 - alpha, axis=0)
    return lower, upper

def spline_fit(erp):
    """固定 50 ms 结点的三次样条；约束自由度，避免近似逐点插值。"""
    mask=(TIMES>=50)&(TIMES<=750)
    t=TIMES[mask]
    y=erp[mask]
    knots=np.arange(100.,750.,50.)
    spline=LSQUnivariateSpline(t,y,knots,k=3)
    fitted=spline(TIMES)
    residual=erp-fitted
    win=np.flatnonzero(WINDOW)
    segment=fitted[win]
    prominence=max(1.,.10*float(np.ptp(segment)))
    peaks,properties=find_peaks(segment,prominence=prominence)
    peaks=peaks[(peaks>2)&(peaks<len(segment)-3)&(segment[peaks]>0)]
    if len(peaks):
        best=peaks[np.argmax(segment[peaks])]
        peak=float(segment[best]);lat=float(TIMES[win[best]])
        status='窗内局部正峰，非生理确认'
    else:
        peak=np.nan;lat=np.nan;status='无合格窗内局部正峰'
    return fitted,dict(fit_family='fixed_knot_cubic_spline',knot_spacing_ms=50.,
        fit_RMSE_50_750=float(np.sqrt(np.mean(residual[mask]**2))),
        fit_RMSE_250_500=float(np.sqrt(np.mean(residual[WINDOW]**2))),
        positive_mean=float(np.mean(np.maximum(segment,0))),
        positive_area_unit_ms=float(np.trapezoid(np.maximum(segment,0),TIMES[WINDOW])),
        local_positive_peak=peak,local_peak_latency_ms=lat,peak_status=status)

```

问题二建模代码	编程语言：Python
<pre> # 问题二：形状编码、神经群体动力学与方向判别 from __future__ import annotations from io import BytesIO from pathlib import Path from zipfile import ZipFile import numpy as np from PIL import Image from scipy.ndimage import gaussian_filter, label, maximum_filter DOCX = '服务于脑机接口与精神性疾病诊断的脑电图计算模型.docx' MEDIA = 'word/media/image12.png' def cue_images(root: Path): with ZipFile(Path(root) / DOCX) as archive: raw = archive.read(MEDIA) image = np.asarray(Image.open(BytesIO(raw)).convert('RGB')).astype(np.float64) blue = np.maximum(0., image[:, :, 2] - np.maximum(image[:, :, 0], image[:, :, 1])) / 255 mask = blue > .2 components, count = label(mask) pieces = sorted([(int(np.sum(components == j)), j) for j in range(1, count + 1)], reverse=True) if len(pieces) < 2 or pieces[0][0] < 80 or pieces[1][0] < 40: raise ValueError('Cannot identify the triangle and fixation circle in the problem image') top, bottom = sorted((pieces[0][1], pieces[1][1]), key=lambda j: np.mean(np.nonzero(components == j)[0])) triangle = np.where(components == top, blue, 0.) circle = np.where(components == bottom, blue, 0.) center_x = int(round(np.mean(np.nonzero(circle)[1]))) left_triangle = np.zeros_like(triangle) ys, xs = np.nonzero(triangle) reflected_x = 2 * center_x - xs inside = (reflected_x >= 0) & (reflected_x < triangle.shape[1]) left_triangle[ys[inside], reflected_x[inside]] = triangle[ys[inside], xs[inside]] if not np.isclose(triangle.sum(), left_triangle.sum(), rtol=0, atol=1e-10): raise ValueError('Mirroring lost triangle pixels') return dict(right=triangle + circle, left=left_triangle + circle, right_triangle=triangle, left_triangle=left_triangle, circle=circle, center_x=center_x, source_size=image.shape[:2], source_media=MEDIA) def oriented_edges(image: np.ndarray) -> np.ndarray: smooth = gaussian_filter(image, .8) gx = gaussian_filter(smooth, 1., order=(0, 1)) gy = gaussian_filter(smooth, 1., order=(1, 0)) angles = np.arange(8) * np.pi / 8 return np.stack([np.abs(gx * np.cos(a) + gy * np.sin(a)) for a in angles]) def sites(template: np.ndarray, n: int = 18) -> list[tuple[int, int, int]]: response = oriented_edges(template) peak = response.max(0) ranked = np.argsort(peak.ravel())[::-1] selected = [] width = template.shape[1] </pre>	

```

for flat in ranked:
    y, x = divmod(int(flat), width)
    if peak[y, x] < .025 * peak.max():
        break
    if any((y - yy) ** 2 + (x - xx) ** 2 < 9 for yy, xx, _ in selected):
        continue
    orientation = int(np.argmax(response[:, y, x]))
    selected.append((y, x, orientation))
    if len(selected) == n:
        break
if len(selected) < n:
    raise ValueError('Too few template edge sites')
return selected

def shape_inputs(cues: dict):
    names = ('left', 'right')
    triangle = (cues['left_triangle'], cues['right_triangle'])
    templates = [sites(item) for item in triangle]
    # One-pixel tolerance; two pixels would pool across much of this 13-pixel cue.
    pooled = [maximum_filter(oriented_edges(item), size=(1, 3, 3))
               for item in triangle]
    activation = np.zeros((2, 2))
    for i, response in enumerate(pooled):
        for j, selected in enumerate(templates):
            values = np.array([response[o, y, x] for y, x, o in selected])
            activation[i, j] = np.exp(np.mean(np.log(values + 1e-5)))
    normalization = np.maximum(np.diag(activation), 1e-10)
    activation /= normalization[None, :]
    # A shared, direction-neutral fixation input is supplied separately.
    common = np.ones(2)
    return dict(inputs=np.column_stack([activation, common]),
                activation=activation, sites=np.array(templates), names=names,
                selectivity=(activation[:, 1] - activation[:, 0]) /
                            (activation[:, 1] + activation[:, 0] + 1e-9))

# ---- 神经群体动力学与留出拟合 ----
from dataclasses import dataclass
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import root
from scipy.special import expit
from scipy.ndimage import maximum_filter
from .stimulus_shape import cue_images, oriented_edges, sites
from .data import TIMES, WEIGHTS, FS, preprocess, means

SCALES = (.75, 1., 1.25)
RECURRENCES = (0., .1, .2)
LAMBDA = np.array([.1, .3, 1., 3., 10., 30., 100.])
CHANNELS = ('Fz', 'F3', 'F4')
STAGES = ('早期视觉', '形状整合', '额区响应')

def encode_shapes(root_path):
    cue = cue_images(root_path)
    left, right = cue['left_triangle'], cue['right_triangle']

```

```

yy, xx = np.nonzero(right)
missing = right.copy()
missing[:, int(np.quantile(xx, .70)):] = 0
# Permute narrow vertical strips within the triangle bounding box only.
scrambled = right.copy()
lo, hi = int(xx.min()), int(xx.max()) + 1
strips = np.array_split(np.arange(lo, hi), 3)
scrambled[:, lo:hi] = right[:, np.concatenate([strips[1], strips[2], strips[0]])]
images = np.stack([left, right, missing, scrambled])
templates = [sites(left), sites(right)]
response = np.stack([maximum_filter(oriented_edges(im), size=(1, 3, 3)) for im in images])
activation = np.array([[np.exp(np.mean(np.log([r[o, y, x] + 1e-5
        for y, x, o in positions]))) for positions in templates]
        for r in response])
activation /= np.maximum(np.diag(activation[:2]), 1e-10)[None, :]
return dict(images=images, inputs=np.c_[activation, np.ones(4)],
        names=np.array(['left', 'right', 'missing_tip', 'scrambled']),
        sites=np.array(templates), circle=cue['circle'],
        source_media=cue['source_media'])

def simulate(inputs, scale=1., recurrence=.1, duration=52/256, max_step=.002):
    """LGN (3), E/I (3 stages x 3 groups), two-stage synapses (36 states).

    Time constants and couplings are modelling assumptions. Excitatory and
    inhibitory postsynaptic filters give  $q = vE - .7 vI$ , not raw spike rates.
    """
    inputs = np.atleast_2d(inputs)
    tau_e = scale * np.array([.020, .040, .080])[:, None]
    tau_i = 2 * tau_e
    tau_se = scale * np.array([.025, .050, .100])[:, None]
    tau_si = 3 * tau_se
    W = (np.ones((3, 3)) - np.eye(3)) / 2
    def neural(g, e, i):
        drive = np.vstack([g, 3 * (e[:-1] - resting_e[:-1])])
        de = (-e + expit(e - i + recurrence * (e @ W.T) + drive - 2)) / tau_e
        di = (-i + expit(e - i - 2)) / tau_i
        return de, di
    # Resting state is identical in all stages; feedforward deviations vanish.
    eq = root(lambda z: np.r_-z[:3] + expit(z[:3]-z[3:] + recurrence*W@z[:3]-2),
        -z[3:] + expit(z[:3]-z[3:]-2)], np.full(6, .12))
    if not eq.success:
        raise RuntimeError('Resting-state solution failed')
    resting_e = np.tile(eq.x[:3], (3, 1))
    resting_i = np.tile(eq.x[3:], (3, 1))
    state0 = np.r_[np.zeros(3), resting_e.ravel(), resting_i.ravel(), np.zeros(36)]
    def derivative(t, state, feature):
        g = state[:3]; e = state[3:12].reshape(3, 3); i = state[12:21].reshape(3, 3)
        ue, ve, ui, vi = state[21:].reshape(4, 3, 3)
        de, di = neural(g, e, i)
        drive = 3 * feature if 0 <= t < duration else np.zeros(3)
        return np.r_[(-g + drive)/(.020*scale), de.ravel(), di.ravel(),
            ((e-resting_e-ue)/tau_se).ravel(), ((ue-ve)/tau_se).ravel(),
            ((i-resting_i-ui)/tau_si).ravel(), ((ui-vi)/tau_si).ravel()]
    eps = 1e-6

```

```

jac = np.column_stack([(derivative(-1, state0 + eps*v, np.zeros(3)) -
                        derivative(-1, state0 - eps*v, np.zeros(3)))/(2*eps)
                        for v in np.eye(len(state0))])
eigmax = float(np.linalg.eigvals(jac).real.max())
if eigmax >= 0:
    raise RuntimeError('Unstable model configuration')
t = np.arange(-250, 4001)/1000
states = []
# Integrate separately at stimulus discontinuities, avoiding adaptive-step skips.
for feature in inputs:
    state = state0.copy(); parts = []
    for start, stop in ((-25, 0.), (0., duration), (duration, 4.)):
        idx = np.flatnonzero((t >= start) & ((t < stop) if stop < 4 else (t <= stop)))
        sol = solve_ivp(lambda tt, z: derivative(tt, z, feature), (start, stop), state,
                        dense_output=True, max_step=max_step, rtol=1e-7, atol=1e-9)
        if not sol.success:
            raise RuntimeError(sol.message)
        parts.append(sol.sol(t[idx])); state = sol.y[:, -1]
    states.append(np.concatenate(parts, axis=1))
states = np.stack(states)
syn = states[:, 21:].reshape(len(inputs), 4, 3, 3, len(t))
q = syn[:, 1] - .7*syn[:, 3]
return dict(t=t, state=states, q=q, eigenmax=eigmax, scale=scale,
            recurrence=recurrence, duration=duration)

def observation_sources(sim):
    """Apply the EEG observation filter at the actual 256-Hz sampling rate.

    Zero padding represents resting activity, not extra fitted trials. Raw
    neural states remain causal; the observed zero-phase-filtered waves need not.
    """
    grid = np.arange(-32*FS, 36*FS)/FS
    q = sim['q'].reshape(len(sim['q']), 9, -1)
    full = np.array([np.interp(grid, sim['t'], s, left=0, right=0) for s in cond] for cond in q])
    filtered = preprocess(full)
    ix = np rint((TIMES-grid[0])*FS).astype(int)
    sampled = filtered[..., ix]
    sampled -= np.median(sampled[..., TIMES < 0], axis=-1, keepdims=True)
    return sampled.transpose(0, 2, 1)

def common_delta(a):
    return np.stack([a.mean(axis=0), a[1]-a[0]])

def condition_waves(cd):
    return np.stack([cd[0]-.5*cd[1], cd[0]+.5*cd[1]])

def select_contrast_shrinkage(bank, projected, ds, train, factors=(0., .25, .5, 1.)):
    """Select contrast attenuation using training blocks only.

    Each inner template is itself chosen without its validation block. Common
    response is unchanged. This is predictive regularization, not a new source.

```

```

"""
errors=np.zeros(len(factors)); records=[]
for held in np.unique(ds.blocks[train]):
    inner=train & (ds.blocks!=held); val=train & (ds.blocks==held)
    best,_=bank.select_prepared(projected,ds.y,ds.blocks,inner)
    coef=bank.inverse[best]@bank.prepared_stats(projected,ds.y,inner)[best]
    pred=bank.predict(coef,best); observed=means(ds.x[val],ds.y[val])
    for j,factor in enumerate(factors):
        cd=common_delta(pred); cd[1]*=factor
        loss=delta_metrics(observed,condition_waves(cd))['delta_MSE']
        errors[j]+=loss
    records.append(dict(inner_held=int(held),template_candidate=int(best),
        train_ids=ds.ids[inner].tolist(),validation_ids=ds.ids[val].tolist()))
errors/=len(records)
return float(factors[int(np.argmin(errors))]),errors,records
# ---- 机制增强特征与块留出判别 ----
import numpy as np
import pandas as pd
from scipy.fft import dct
from sklearn.metrics import roc_auc_score

from q2model.data import NAMES, TIMES, independent_data, modes, save_csv, save_json

ROOT = Path(__file__).resolve().parents[1]
POST_MASK = (TIMES >= .05) & (TIMES < .75)
N_DCT = 6
SHRINK = .8 # fixed isotropic shrinkage of within-class covariance
PRE_TO_POST RIDGE_FACTOR = 1.0 # penalty = number of training trials
MODELS = ('past_only', 'post_erp', 'post_given_past')
MODES = ('common', 'midline', 'lateral')

def dct_features(x: np.ndarray, mask: np.ndarray | None = None) -> np.ndarray:
    """Orthonormal low-frequency coefficients, 3 scalp modes x 6 time modes."""
    if x.ndim != 3 or x.shape[1] != 3:
        raise ValueError('Expected trials x Fz/F3/F4 x time')
    segment = x if mask is None else x[:, mask]
    if segment.shape[-1] < N_DCT:
        raise ValueError('Not enough time samples for DCT')
    return dct(modes(segment), type=2, norm='ortho', axis=-1)[..., :N_DCT].reshape(len(x), -1)

def standardize(train: np.ndarray, test: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    center = train.mean(axis=0)
    scale = np.maximum(train.std(axis=0), 1e-8)
    return np.clip((train - center) / scale, -8, 8), np.clip((test - center) / scale, -8, 8)

def remove_past_prediction(pre_train: np.ndarray, pre_test: np.ndarray,
    post_train: np.ndarray, post_test: np.ndarray
) -> tuple[np.ndarray, np.ndarray]:
    """Label-blind, train-only ridge regression of post waveform on past EEG."""
    if pre_train.shape[0] != post_train.shape[0] or pre_test.shape[0] != post_test.shape[0]:
        raise ValueError('Paired EEG features required')

```

```

penalty = PRE_TO_POST RIDGE_FACTOR * len(pre_train)
transform = np.linalg.solve(pre_train.T @ pre_train + penalty * np.eye(pre_train.shape[1]),
                             pre_train.T @ post_train)
return post_train - pre_train @ transform, post_test - pre_test @ transform

def erp_score(train: np.ndarray, labels: np.ndarray, test: np.ndarray) -> np.ndarray:
    """Shrinkage-whitened right-minus-left ERP template, balanced prior."""
    if set(np.unique(labels)) != {-1, 1}:
        raise ValueError('Both cue directions required in training')
    left, right = train[labels == -1], train[labels == 1]
    mu_left, mu_right = left.mean(axis=0), right.mean(axis=0)
    residual = np.vstack((left - mu_left, right - mu_right))
    cov = residual.T @ residual / max(len(train) - 2, 1)
    identity_scale = np.trace(cov) / cov.shape[0]
    regularized = (1 - SHRINK) * cov + SHRINK * max(identity_scale, 1e-6) * np.eye(cov.shape[0])
    weight = np.linalg.solve(regularized, mu_right - mu_left)
    return (test - .5 * (mu_left + mu_right)) @ weight

def load_datasets() -> dict[str, dict[str, np.ndarray]]:
    out = {}
    for index, key in enumerate(NAMES):
        ds = independent_data(ROOT, key)
        out[key] = dict(pre=dct_features(ds.prestim_only), post=dct_features(ds.x, POST_MASK),
                        y=ds.y, blocks=ds.blocks, ids=ds.ids,
                        global_blocks=ds.blocks + 5 * index)
    return out

def decode_one(data: dict[str, np.ndarray], labels: np.ndarray,
               return_features: bool = False) -> tuple[dict[str, np.ndarray], dict[str, np.ndarray]]:
    """Five contiguous-block folds; no outer-test labels enter any fitting step."""
    scores = {name: np.full(len(labels), np.nan) for name in MODELS}
    features = {name: np.full_like(data['post'], np.nan) for name in MODELS} if return_features else {}
    for held in np.unique(data['blocks']):
        train, test = data['blocks'] != held, data['blocks'] == held
        pre_train, pre_test = standardize(data['pre'][train], data['pre'][test])
        post_train, post_test = standardize(data['post'][train], data['post'][test])
        residual_train, residual_test = remove_past_prediction(
            pre_train, pre_test, post_train, post_test)
        fold = {'past_only': (pre_train, pre_test), 'post_erp': (post_train, post_test),
               'post_given_past': (residual_train, residual_test)}
        for name, (x_train, x_test) in fold.items():
            scores[name][test] = erp_score(x_train, labels[train], x_test)
            if return_features:
                features[name][test] = x_test
    if any(not np.isfinite(arr).all() for arr in scores.values()):
        raise ValueError('Incomplete outer-fold predictions')
    return scores, features

def metrics(y: np.ndarray, scores: np.ndarray) -> dict[str, float]:
    pred = np.where(scores >= 0, 1, -1)
    return dict(n=int(len(y)), BA=float(.5 * ((pred[y == -1] == -1).mean() +

```

```

        (pred[y == 1] == 1).mean()),
    AUC=float(roc_auc_score(y, scores)),
    recall_left=float((pred[y == -1] == -1).mean()),
    recall_right=float((pred[y == 1] == 1).mean()))

def evaluate(data: dict[str, dict[str, np.ndarray]],
            labels: dict[str, np.ndarray], return_features: bool = False):
    score_map, feature_map = {}, {}
    for key, group in data.items():
        score_map[key], feature_map[key] = decode_one(group, labels[key], return_features)
    rows = []
    for key in ('pooled', *NAMES):
        for name in MODELS:
            yy = np.concatenate([labels[k] for k in NAMES]) if key == 'pooled' else labels[key]
            ss = np.concatenate([score_map[k][name] for k in NAMES]) if key == 'pooled' else score_map[key][name]
            rows.append(dict(dataset=key, model=name, **metrics(yy, ss)))
    return pd.DataFrame(rows), score_map, feature_map

def permute_within_blocks(data: dict[str, dict[str, np.ndarray]],
                          rng: np.random.Generator) -> dict[str, np.ndarray]:
    labels = {}
    for key, group in data.items():
        labels[key] = group['y'].copy()
        for block in np.unique(group['blocks']):
            index = np.flatnonzero(group['blocks'] == block)
            labels[key][index] = rng.permutation(labels[key][index])
    return labels

def block_bootstrap(data, score_map, n_boot: int, seed: int) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    rows = []
    for dataset in ('pooled', *NAMES):
        for model in MODELS:
            vals = []
            for _ in range(n_boot):
                yy, ss = [], []
                for key in NAMES if dataset == 'pooled' else (dataset,):
                    group = data[key]
                    blocks = np.unique(group['blocks'])
                    chosen = rng.choice(blocks, len(blocks), replace=True)
                    index = np.concatenate([np.flatnonzero(group['blocks'] == b) for b in chosen])
                    yy.append(group['y'][index]); ss.append(score_map[key][model][index])
                y = np.concatenate(yy); score = np.concatenate(ss)
                pred = np.where(score >= 0, 1, -1)
                vals.append(float(.5 * ((pred[y == -1] == -1).mean() +
                                       (pred[y == 1] == 1).mean())))
            low, high = np.quantile(vals, [.025, .975])
            rows.append(dict(dataset=dataset, model=model, BA_low=float(low), BA_high=float(high),
                           bootstraps=n_boot, resampling_unit='contiguous_time_block'))
    return pd.DataFrame(rows)

```

```

# 问题三：事件审计、认知状态拟合与决策模拟
import numpy as np
from .common import FS, TIME, ROOT, Trials, load_raw, runs, causal_filter, quality

def audit_events(key, data):
    cues, markers = runs(data[7]), runs(data[8])
    rows = []
    for i, (on, off, cue) in enumerate(cues):
        nxt = cues[i+1][0] if i+1 < len(cues) else data.shape[1]
        ev = [(s, e, v) for s, e, v in markers if on <= s < nxt]
        platforms = [(s, e, v) for s, e, v in ev if abs(v) == 1]
        clicks = [(s, e, v) for s, e, v in ev if abs(v) == 2]
        ts, te, sign = platforms[0] if len(platforms) == 1 else (None, None, None)
        target_valid = ts is not None and on < ts < nxt
        marker_valid = len(clicks) <= 1
        click = clicks[0][0] if len(clicks) == 1 else None
        if click is not None and (not target_valid or not ts < click < nxt):
            marker_valid = False
        response, status = None, 'unobserved'
        if marker_valid and click is not None:
            response, status = click, 'explicit_click'
        elif marker_valid and key.endswith('1') and target_valid and ts < te < nxt:
            response, status = te, 'platform_end_proxy'
        row = dict(dataset=key, trial_id=i+1, block=i//20, cue=int(cue),
            cue_sample=on, cue_end_sample=off, cue_duration_s=(off-on)/FS,
            next_cue_sample=nxt, record_stop_sample=data.shape[1],
            target_sample=ts, target_s=(ts-on)/FS if target_valid else None,
            target_status='platform_onset_hypothesis' if target_valid else 'unknown',
            platform_end_sample=te, platform_sign=sign,
            click_sample=click, click_side=int(np.sign(clicks[0][2])) if len(clicks)==1 else None,
            response_sample=response, response_s=(response-on)/FS if response is not None else None,
            rt_s=(response-ts)/FS if response is not None else None, response_status=status,
            correctness=None, correctness_reason='target_ground_truth_unavailable',
            timeout=None, timeout_reason='experimental_deadline_unavailable',
            n_platforms=len(platforms), n_clicks=len(clicks),
            cue_platform_disagree=bool(sign!=cue) if sign is not None else None,
            valid_events=bool(target_valid and marker_valid),
            event_issue='none' if target_valid and marker_valid else 'ambiguous_or_invalid_markers')
        rows.append(row)
    return rows

def window_policy(event, horizon=2., gap=.1):
    """All correctness values are eligible. Administrative cutoff != timeout.

    Missing-marker records have a finite EEG window but are not automatically
    valid right-censored behavior: ascertainment of the response is unknown.
    """
    if horizon <= 0 or gap < 0:
        raise ValueError('Invalid window policy')
    e = dict(event)
    e.update(horizon_s=float(horizon), gap_s=gap, retained=False,
        end_s=None, behavior_duration_s=None, behavior_event=None,
        behavior_survival_eligible=False, eeg_administrative_cutoff=False)
    if not e['valid_events']:
        e['window_status']='unusable_events'
    return e

```

```

target = e['target_s']
administrative = target+horizon
available = min((e['next_cue_sample']-e['cue_sample'])/FS,
                (e['record_stop_sample']-e['cue_sample'])/FS, float(TIME[-1]))
bound = min(administrative, available)
response = e['response_s']
# Gap is attached to an actual/proxy response, not subtracted from censoring.
e['end_s'] = min(bound, response-gap) if response is not None else bound
e['eeg_administrative_cutoff'] = response is None or bound < response-gap
e['window_status'] = ('response_minus_gap' if not e['eeg_administrative_cutoff']
                     else 'bounded_observation')
e['bound_reason'] = 'configured_horizon' if administrative <= available else 'record_or_next_trial'
duration = max(0., bound-target)
if e['response_status']=='explicit_click':
    e['behavior_survival_eligible']=duration>0
    e['behavior_event']=int(response <= bound)
    e['behavior_duration_s']=min(response-target,duration)
    e['behavior_status']='observed_click' if e['behavior_event'] else 'administratively_right_censored'
elif e['response_status']=='platform_end_proxy':
    e['behavior_status']='proxy_only_not_observed_click'
else:
    e['behavior_status']='unknown_response_or_missing_marker'
if e['end_s'] <= target:
    e['window_status']='no_posttarget_observation'
return e

def build_trials(key, horizon=2., gap=.1, raw=None, root=ROOT):
    data = load_raw(key, root)[0] if raw is None else raw
    audited = audit_events(key,data)
    ons=[e['cue_sample'] for e in audited]
    edges=[0]+[(ons[j-1]+ons[j])/2 for j in (20,40,60,80)]+[data.shape[1]]
    pieces={b:causal_filter(data[:3,a:z]) for b,(a,z) in enumerate(zip(edges[:-1],edges[1:]))}
    kept=[];arrays=[];scales=[];rows=[]
    for event in audited:
        e=window_policy(event,horizon,gap);b=e['block'];on=e['cue_sample'];a,z=edges[b:b+2]
        e.update(filter_start_sample=a,filter_stop_sample=z,filter_mode='causal',reason='invalid_events')
        if e['end_s'] is not None and e['end_s']>e['target_s']:
            end=on+int(np.ceil(e['end_s']*FS));length=end-(on-64)
            e.update(epoch_start_sample=on-64,epoch_end_exclusive=end)
            if on-64<a+24*FS or end>z-24*FS:
                e['reason']='block_boundary_24s_buffer'
            else:
                epoch=pieces[b][:,on-64-a:end-a].copy()
                epoch=np.median(epoch[:,64:],axis=1,keepdims=True)
                bad,measures=quality(epoch,data[:3,on-64:end],np.arange(length)>=64)
                e.update(measures,retained=not bad,reason='quality' if bad else 'retained')
            if not bad:
                x=np.full((len(TIME),3),np.nan);x[:length]=epoch.T
                arrays.append(x);kept.append(e)
                scales.append(np.maximum(1.4826*np.median(abs(epoch[:,64:])),axis=1,1.))
        rows.append(e)
    if not kept: raise ValueError(f'No valid EEG trials for {key}')
    return Trials(key,kept,np.stack(arrays),np.stack(scales),'causal',gap),rows

# ---- 记忆与认知状态 ----
from dataclasses import dataclass, asdict
from functools import lru_cache
from itertools import product
import numpy as np
from scipy.signal import sosfilt

```

```

from q2.q2model.generative import simulate, encode_shapes
from .common import FS, TIME, ROOT, filter_sos

@lru_cache(maxsize=1)
def shape_inputs():
    return encode_shapes(ROOT)['inputs'][:2]

@dataclass(frozen=True)
class Candidate:
    model: str
    tau_m: float = 1.
    tau_p: float = .3
    target_duration: float = .2
    direction_penalty: float = 1.
    delay: float = .05

    def record(self):
        return asdict(self)

def candidates():
    ans=[]
    for duration,ratio in product((.1,.2,.4),(1.,10.)):
        ans.append(Candidate('N0',target_duration=duration,direction_penalty=ratio))
        for tp in (.1,.3,.6):
            for model in ('N1','N3'):
                ans.append(Candidate(model,tau_p=tp,target_duration=duration,direction_penalty=ratio))
            for tm in (.5,1.,2.,4.):
                for model in ('N2','N2_no_projection','N2_no_retrieval'):
                    ans.append(Candidate(model,tm,tp,duration,ratio))
    ans.append(Candidate('B0'))
    return ans

def stimulus_key(e):
    return int(e['cue']),int(round(e['cue_duration_s']*FS)),int(round(e['target_s']*FS))

@lru_cache(maxsize=24)
def pulse(duration, common_target=False):
    inputs=np.array([[0.,0.,1.]]) if common_target else shape_inputs()
    result=simulate(inputs,duration=duration,max_step=1/FS)
    # Retain Q2's last visual stage. Its source is sampled on a fixed horizon.
    q=result['q'][:,2]
    return result['t'],q

@lru_cache(maxsize=128)
def visual_parts(key,duration):
    cue,ns,ts=key
    t,q=pulse(ns/FS)
    left=np.stack([np.interp(TIME,t,a,left=0,right=0) for a in q[0]],axis=1)
    right=np.stack([np.interp(TIME,t,a,left=0,right=0) for a in q[1]],axis=1)
    # Common activity and antisymmetric shape contrast have distinct readouts.
    common=(left.mean(1)+right.mean(1))/2
    contrast=(right[:,1]-left[:,1])/2
    tc,qt=pulse(duration,True)

```

```

target=np.interp((TIME-ts)/FS,tc,qt[0,2],left=0,right=0)
return np.stack([common,cue*contrast,target],axis=1)

def lowpass(x,tau):
    # Exponential Euler: state at t uses input at t-1, hence is causal.
    from scipy.signal import lfilter
    a=np.exp(-1/(FS*tau))
    return lfilter([0.,1-a],[1,-a],x,axis=0)

def states(e,c):
    v=visual_parts(stimulus_key(e),c.target_duration)
    m=lowpass(v[:,2],c.tau_m) # Target activity NEVER enters stored cue memory.
    delayed=np.stack([np.interp((TIME-c.delay,TIME,m[:,j]),left=0,right=0) for j in range(2)],axis=1)
    gate=(TIME>=e['target_s'][:,None])
    gain=0. if c.model=='N2_no_retrieval' else 1.
    drive=v.copy()
    if c.model.startswith('N2'):
        drive[:,2]+=gain*gate*delayed
    p=lowpass(drive,c.tau_p)
    return v,m,p

@lru_cache(maxsize=4096)
def neural_design(key,c):
    e=dict(cue=key[0],cue_duration_s=key[1]/FS,target_s=key[2]/FS)
    v,m,p=states(e,c);h=np.zeros((len(TIME),9));h[:,3]=v
    if c.model=='B0':
        h[:,0]=0
    elif c.model=='N1':
        h[:,6:9]=p
    elif c.model.startswith('N2'):
        if c.model!='N2_no_projection':h[:,3:5]=m
        h[:,6:9]=p
    elif c.model=='N3':
        t=np.maximum(TIME,0);a=np.maximum(TIME-e['target_s'],0)
        s=np.stack([(TIME>=0)*(1-np.exp(-t/c.tau_p)),t/(1+t),
                    (TIME>=e['target_s'])*(1-np.exp(-a/c.tau_p))],axis=1)
        h[:,3:6]=s;h[:,6:9]=s*e['cue']
    h=sosfilt(filter_sos(),h,axis=0)
    h=np.median(h[TIME<0],axis=0)
    return h.astype(np.float32)
# ---- 分块选择与稳健拟合 ----
import numpy as np
from scipy.linalg import solve
from .model import design_bank, penalties

LAMBDA_S=np.array([.001,.01,.1,1,10.])

class Bank:
    def __init__(self,ds,params,context=False,design_function=None):
        self.ds=ds;self.params=params;self.context=context;self.dim=18 if context else 9
        self.design=design_bank if design_function is None else design_function
        self.fit_cache={};self.robust_cache={};self.weights=ds.weights
        self.penalty=np.stack([penalties(c,self.dim) for c in params])
        n=len(params);d=self.dim

```

```

self.G=np.zeros((5,n,d,d));self.B=np.zeros((5,n,d,3));self.Y=np.zeros((5,3));self.N=np.zeros(5)
# A design can include pre-cue context, so sufficient statistics are
# accumulated per trial (never grouped while dropping its context).
for i,e in enumerate(ds.events):
    block=e['block'];h=self.design(params,e,context=context).astype(float)
    w=self.weights[i];x=np.nan_to_num(ds.x[i])
    self.G[block]+=np.matmul(h.transpose(0,2,1),h*w[None,:,None])
    self.B[block]+=np.matmul(h.transpose(0,2,1),w[:,None]*x)
    self.Y[block]+=np.sum(w[:,None]*x,axis=0)
    self.N[block]+=1

def ridge_all(self,blocks):
    ix=list(blocks);count=self.N[ix].sum()
    if count<=0:raise ValueError('Empty training blocks')
    g=self.G[ix].sum(0)/count;b=self.B[ix].sum(0)/count
    scale=np.maximum(np.sqrt(np.diagonal(g,axis1=1,axis2=2)),1e-8)
    gn=g/scale[:,None]/scale[:,None];bn=b/scale[:,None]
    systems=gn[None]+LAMBDA[:,None,None,None]*np.eye(self.dim)[None,None]*self.penalty[None,:,None,:]
    coef=np.linalg.solve(systems,np.broadcast_to(bn,systems.shape[:2]+(self.dim,3)))
    return coef/scale[None,:,None],scale,np.maximum(self.Y[ix].sum(0)/count,1.)

def validation_errors(self,coef,block,channel_var):
    # lambda, candidate, channel
    err=(np.einsum('lkpc,kpc->lkc',coef,self.G[block],coef,optimize=True)
        -2*np.einsum('lkpc,kpc->lkc',coef,self.B[block],optimize=True)+self.Y[block])
    return np.mean(err/channel_var[None,None,:],axis=-1)/self.N[block]

def matrix(self,indices,candidate_index):
    hs=[];ys=[];ws=[]
    c=self.params[candidate_index]
    for i in indices:
        h=self.design([c],self.ds.events[i],context=self.context)[0].astype(float)
        mask=self.weights[i]>0
        hs.append(h[mask]);ys.append(self.ds.x[i,mask]);ws.append(self.weights[i,mask])
    return np.concatenate(hs),np.concatenate(ys),np.concatenate(ws)/len(indices)

def robust_fit(self,blocks,ci,lam,iterations=7):
    cache_key=(tuple(sorted(blocks)),ci,lam,iterations)
    if cache_key in self.robust_cache:return self.robust_cache[cache_key]
    idx=np.flatnonzero(np.isin(self.ds.blocks,list(blocks)))
    h,y,w=self.matrix(idx,ci)
    source_scale=np.maximum(np.sqrt(np.sum(w[:,None]*h*h,axis=0)),1e-8)
    channel_scale=np.maximum(np.sqrt(np.sum(w[:,None]*y*y,axis=0)),1.)
    z=h/source_scale;target=y/channel_scale
    gram=z.T@(w[:,None]*z)
    co=solve(gram+lam*np.diag(self.penalty[ci]),z.T@(w[:,None]*target),assume_a='pos')
    # Huber M-estimation on trial/stage-balanced, channel-normalized residuals.
    convergence=[]
    for _ in range(iterations):
        residual=target-z@co;robust=np.minimum(1.,1.5/np.maximum(abs(residual),1e-12))
        new=np.empty_like(co)
        for channel in range(3):
            wc=w*robust[:,channel]
            new[:,channel]=solve(z.T@(wc[:,None]*z)+lam*np.diag(self.penalty[ci]),z.T@(wc*target[:,channel]),assume_a=
'pos')
        change=float(np.max(abs(new-co)));convergence.append(change);co=new
    if change<1e-6:break
    result=co/source_scale[:,None]*channel_scale[None,:]
```

```

        edf=float(np.trace(np.linalg.solve(gram+lam*np.diag(self.penalty[ci]),gram)))
        singular=np.linalg.svd(np.sqrt(w[:,None])*z,compute_uv=False)
        answer=result,dict(source_scale=source_scale,channel_scale=channel_scale,edf=edf,
                            design_singular_values=singular,irls_changes=convergence)
        self.robust_cache[cache_key]=answer
    return answer
# ---- 首达边界决策过程 ----
from dataclasses import replace
import numpy as np
from .common import FS, TIME
from .model import Candidate, states

REFERENCE=Candidate('N2',tau_m=2.,tau_p=.3,target_duration=.2)
EVENT=dict(cue=1,cue_duration_s=52/FS,target_s=568/FS)
SCENARIOS=(
    ('nominal','基准',REFERENCE,1.5,.6,2.),
    ('weak_memory','短保持',replace(REFERENCE,tau_m=.5),1.5,.6,2.),
    ('retrieval_off','关闭提取',replace(REFERENCE,model='N2_no_retrieval'),1.5,.6,2.),
    ('high_noise','高噪声',REFERENCE,1.5,1.3,2.),
    ('low_evidence','低证据增益',REFERENCE,.15,.6,2.),
    ('short_deadline','短仿真截止',REFERENCE,1.5,.6,1.2),
)

def aligned_evidence(candidate,event=EVENT,deadline=2.,dt=1/FS):
    t=np.arange(int(np.floor(deadline/dt))*dt)
    if not len(t) or event['target_s']+deadline>TIME[-1]:
        raise ValueError('Decision horizon exceeds neural simulation')
    __,p=states(event,candidate)
    return t,event['cue']*np.interp(event['target_s']+t,TIME,p[:,1])

def evidence_normalization():
    __,a=aligned_evidence(REFERENCE)
    return max(float(np.max(abs(a))),1e-10)

def target_match(cues,layouts):
    """Signed right-minus-left similarity; no correctness labels are accepted.

    Shapes are encoded by the same Q2 image encoder. This extension represents
    simulated Task-2 layouts only; real per-trial layouts remain unavailable.
    """
    from .model import shape_inputs
    features=shape_inputs()[::2]
    features=features/np.maximum(np.linalg.norm(features,axis=1,keepdims=True),1e-10)
    cues=np.asarray(cues);layouts=np.asarray(layouts)
    if layouts.shape!=(len(cues),2) or not np.isin(layouts,[-1,1]).all():
        raise ValueError('Supply left/right target shapes for every simulated trial')
    memory=features[(cues==1).astype(int)]
    targets=features[(layouts==1).astype(int)]
    similarities=np.einsum('nd,nkd->nk',memory,targets)
    scale=max(1-float(features[0]@features[1]),1e-10)
    return (similarities[:,1]-similarities[:,0])/scale

def simulate(candidate=REFERENCE,gamma=1.5,sigma=.6,deadline=2.,n=6000,
            seed=202609263,dt=1/FS,boundary=1.,event=EVENT,evidence_mode='conditional'):

```

```

if n%4 or n<4 or sigma<0 or boundary<=0:
    raise ValueError('Use balanced four-way simulation and valid noise/boundary')
time,evidence=aligned_evidence(candidate,event,deadline,dt)
horizon=len(time)*dt
drift=gamma*evidence/evidence_normalization()
# Ground-truth layout exists ONLY here. Cue shape and target side are distinct.
cue=np.tile([-1,-1,1,1],n//4);correct_side=np.tile([-1,1,-1,1],n//4)
layouts=np.column_stack([np.tile([-1,1,-1,1],n//4),np.tile([1,-1,1,-1],n//4)])
if evidence_mode=='matching':
    direction=target_match(cue,layouts)
    correct_side=np.where(layouts[:,1]==cue,1,-1) # scoring only
elif evidence_mode=='conditional':direction=correct_side.astype(float)
else:raise ValueError('Unknown evidence mode')
rng=np.random.default_rng(seed);z=np.zeros(n);choice=np.zeros(n,int)
decision=np.full(n,np.nan);history=np.zeros((n,len(time)),np.float32)
for j in range(len(time)):
    noise=rng.standard_normal(n) # common random numbers across scenarios
    active=choice==0
    z[active]+=direction[active]*drift[j]*dt+sigma*np.sqrt(dt)*noise[active]
    hit=active&(abs(z)>=boundary)
    choice[hit]=np.sign(z[hit]).astype(int);decision[hit]=(j+1)*dt
    history[:,j]=z
outcome=np.where(choice==0,'unresolved',np.where(choice==correct_side,'correct','error'))
rows=[dict(simulation_id=i+1,cue=int(cue[i]),correct_side=int(correct_side[i]),
    target_left_shape=int(layouts[i,0]) if evidence_mode=='matching' else None,
    target_right_shape=int(layouts[i,1]) if evidence_mode=='matching' else None,
    signed_match=float(direction[i]),evidence_mode=evidence_mode,
    choice=int(choice[i]),outcome=outcome[i],decision_time_s=float(decision[i]),
    observation_time_s=float(decision[i]) if choice[i] else horizon,
    event_observed=int(choice[i]!=0),right_censored=bool(choice[i]==0)) for i in range(n)]
observed_times=np.where(choice!=0,decision,horizon)
resolved=np.sort(decision[np.isfinite(decision)])
median_index=int(np.ceil(n/2))-1
survival_median=float(resolved[median_index]) if len(resolved)>median_index else None
summary=dict(n=n,evidence_mode=evidence_mode,tau_m=candidate.tau_m,tau_p=candidate.tau_p,
    neural_model=candidate.model,target_duration=candidate.target_duration,
    gamma=gamma,sigma=sigma,boundary=boundary,deadline_s=horizon,dt=dt,
    correct_rate=float(np.mean(outcome=='correct')),error_rate=float(np.mean(outcome=='error')),
    unresolved_rate=float(np.mean(outcome=='unresolved')),
    median_decision_time_s=float(np.nanmedian(decision)) if np.isfinite(decision).any() else None,
    conditional_median_scope='resolved_trials_only',
    censor_aware_median_s=survival_median,
    restricted_mean_decision_time_s=float(observed_times.mean()),
    interpretation='simulation_only_no_observed_accuracy_fit')
for name in ('correct','error','unresolved'):
    p=summary[name+'_rate'];k=p*n;zz=1.96**2
    center=(p+zz/(2*n))/(1+zz/n);half=1.96*np.sqrt(p*(1-p)/n+zz/(4*n*n))/(1+zz/n)
    summary[name+'_mc_low']=float(center-half);summary[name+'_mc_high']=float(center+half)
examples=[]
for name in ('correct','error','unresolved'):
    examples.extend(np.flatnonzero(outcome==name)[:3])
paths=dict(time=time+dt,drift=drift,evidence=evidence,trajectories=history[examples],
    outcome=outcome[examples],correct_side=correct_side[examples],ids=np.array(examples)+1)

```